

Vue知识卡片

1 Vue 核心心智

1.1 Vue 不只是视图层

定位

这一节先把整章的主心骨立住：Vue 不是一组零散 API，也不只是模板语法，而是一条从编译到渲染、从状态到应用组织的完整链路。

必会结论

1. Vue 既可以渐进式接入页面，也可以承担完整前端应用的工程底座。
2. Vue 的核心不只是组件写法，还包括响应式系统、模板编译、运行时渲染、路由、状态管理和 SSR。
3. 面试里如果只会讲 `ref`、`watch`、生命周期，说明还停留在使用层；能讲清编译期和运行时分工，才算进入工程层。

核心理解

现代 Vue 应用通常不是直接写 `render()`，而是写 `.vue` 单文件组件。开发者看到的是 `template + script + style`，但 Vue 真正工作的链路是：SFC 被编译器拆解，模板被转成渲染函数，运行时执行渲染函数生成 VNode，响应式系统驱动组件更新，渲染器再把差异落到真实 DOM 上。

从架构上看，Vue 至少有四层值得记住：

1. 响应式层：负责状态追踪和副作用触发。
2. 编译层：负责把模板转成 `render`，并提前标记哪些地方可能变化。
3. 运行时层：负责组件实例、VNode、调度、`patch`。
4. 应用层：负责 Router、Pinia、SSR、工程工具链这些真正落地的能力。

常见坑

1. 把 Vue 理解成“带模板语法的视图层库”。
2. 把 Vue 学成 API 清单，学完一堆函数却讲不出更新链路。
3. 以为 Vue 的性能只来自虚拟 DOM，忽略了编译器和响应式系统的配合。

面试展开

如果让我先用一句话定义 Vue，我会说：Vue 是一个以组件为组织单位、以响应式为状态驱动、以编译器和运行时协同优化渲染的现代前端框架。真正把 Vue 讲深，不是背 API，而是把“组件怎么写”“模板怎么编译”“状态怎么触发更新”“DOM 怎么精准更新”串成一条线。

1.2 Vue3 的核心升级

定位

这一节不是列新特性清单，而是解释 Vue 3 到底升级了什么，为什么这些升级对工程和面试都重要。

必会结论

1. Vue 3 的升级不是只换了 Composition API。
2. Vue 3 的提升来自响应式重构、编译优化、运行时更新优化和 TypeScript 友好化的叠加。
3. Vue 3 比 Vue 2 更强，不是单点变快，而是整条链路更现代了。

核心理解

Vue 3 的核心升级可以按四条主线记：

1. 响应式升级：从 `Object.defineProperty` 换到 `Proxy`，对象新增属性、数组索引、`Map`、`Set` 这些边界处理得更统一。
2. 组织方式升级：从以 Options API 为中心，转向更适合大型组件的 Composition API 和 `<script setup>`。
3. 渲染性能升级：模板编译阶段会做静态提升、PatchFlag 标记、Block Tree 优化，运行时更新不再是“无差别全树比较”。
4. 工程能力升级：源码和类型系统更现代，SFC、TS、工具链、SSR 都更顺手。

如果只用一句话概括 Vue 3 的升级，可以说：它把 Vue 从“好用的前端框架”推进成了“编译期和运行时都很成熟的现代工程体系”。

常见坑

1. 把 Vue 3 的核心升级理解成“多了 `setup` 和 `ref`”。
2. 把 Vue 3 性能提升简单归因成“因为用了 `Proxy`”。
3. 只讲 API 风格变化，不讲编译优化和渲染机制变化。

面试展开

更稳的答法是：Vue 3 的核心升级有三层。第一层是响应式模型从 `defineProperty` 升级到 `Proxy`，解决了很多 Vue 2 的先天限制；第二层是组件组织从 Options API 扩展到了 Composition

API，更适合逻辑复用和大型组件维护；第三层是编译器和运行时配合得更紧，编译期提前给运行时提供优化信息，让 patch 更精准，所以 Vue 3 更快不是一个点快，而是整条链路一起优化了。

1.3 一份 SFC 到页面的完整链路

定位

这一节要建立 Vue 学习里最关键的工程心智：`.vue` 文件不是浏览器直接执行的东西，它必须先经过编译，再进入运行时。

必会结论

1. `.vue` 单文件组件本质上是 Vue 的开发时格式，不是浏览器原生格式。
2. SFC 会先经过 `compiler-sfc` 处理，再把模板编译为 render 函数。
3. 页面最终更新，靠的不是模板字符串替换，而是 render 生成的 VNode 和后续 patch。

代码抓手

代码块

```
1 <template>
2   <button @click="count++">{{ count }}</button>
3 </template>
4
5 <script setup lang="ts">
6   import { ref } from 'vue'
7
8   const count = ref(0)
9 </script>
```

核心理解

上面这段代码背后大致会经历这样的链路：

1. `compiler-sfc` 先解析这个 `.vue` 文件，拆出 `template`、`script`、`style`。
2. `script setup` 会被编译成标准组件逻辑。
3. `template` 会被编译成 render 函数，而不是留到运行时做字符串替换。
4. 组件挂载时，运行时执行 `setup()`，再执行 render，生成一棵 VNode 树。
5. 渲染器根据 VNode 创建真实 DOM。
6. 当 `count.value++` 时，响应式系统触发组件重新渲染，生成新 VNode，再和旧 VNode 做 patch，最后更新按钮文本。

这一节最关键的认知是：你写的是 SFC，Vue 真正执行的是“编译后的组件对象和 render 函数”。

常见坑

1. 以为 `.vue` 文件会被浏览器直接理解。
2. 以为模板更新本质上是“把字符串重新拼进 HTML”。
3. 把 `compiler-sfc` 误叫成 `runtime-sfc`。SFC 是编译期能力，不是运行时能力。

面试展开

如果面试官问“一份 Vue 组件最终怎么显示到页面上”，比较稳的回答是：开发阶段我们通常写 SFC，构建时先由 `compiler-sfc` 处理单文件组件，再把模板编译成 render 函数。运行时创建组件实例，执行 `setup`，render 返回 VNode，renderer 把 VNode 挂载为真实 DOM。后续响应式数据变化后，组件 render effect 重新执行，生成新旧 VNode，再通过 patch 把变化同步到 DOM。

1.4 编译期和运行时分工

定位

这一节专门解决一个高频误区：Vue 的性能和心智模型，不能只从运行时看，也不能只从模板语法看，必须理解编译期和运行时的分工。

必会结论

1. Vue 的模板编译不是“翻译语法”这么简单，它会做静态分析并给运行时留下优化提示。
2. Vue 的响应式更新主要发生在运行时，依赖收集和触发更新是在代码执行过程中完成的。
3. Vue 3 的优势就在于：编译期提前做信息提纯，运行时按这些信息精准更新。

核心理解

编译期主要做三件事：

1. 把模板变成 render 函数。
2. 找出静态内容并提升复用，减少重复创建。
3. 标记动态节点，让运行时知道“该看什么，不该看什么”。

运行时主要做四件事：

1. 创建组件实例，执行 `setup`。
2. 在 render 中读取响应式状态，建立依赖关系。
3. 状态变化后重新执行 render，生成新 VNode。
4. 通过 patch 把变化更新到真实 DOM。

所以 Vue 不是“纯运行时框架”，也不是“纯编译时框架”。它的实际路线更像是：编译期负责缩小问题范围，运行时负责在真实场景中完成依赖追踪和更新落地。

常见坑

1. 把 Vue 理解成“虚拟 DOM 框架，所以每次都整棵树 diff”。
2. 只讲 `Proxy`，不讲编译器提前提供的优化信息。
3. 把模板编译和响应式触发混在一起，分不清谁发生在编译时，谁发生在运行时。

面试展开

一个更工程化的表述是：Vue 3 的性能优势本质上来自编译器和运行时协作。编译器会把模板里的静态节点、动态节点、动态子树信息提前编码到生成结果里，运行时就不需要盲目全量对比，而是可以结合响应式触发的组件更新，只检查真正可能变化的那部分内容。也因此，Vue 3 不是简单的虚拟 DOM，而是“带编译时信息的虚拟 DOM”。

1.5 Vue2 到 Vue3 的主线变化

定位

这一节不是做迁移文档，而是帮你抓住 Vue 2 和 Vue 3 在底层思路上的主线差异。

必会结论

1. Vue 3 不是 Vue 2 的语法换皮，而是核心实现和工程表达都更现代。
2. Vue 2 到 Vue 3 的变化，应该按“响应式、组件组织、渲染优化、工程能力”四条主线理解。
3. 迁移细节很多，但面试里更重要的是先讲清主线，再补充 breaking changes。

核心理解

从主线上看，Vue 2 到 Vue 3 的变化可以这样记：

1. 响应式层：从 `Object.defineProperty` 走向 `Proxy`。
2. 组件组织层：从以 `data / methods / computed / watch` 分区为主，走向以“功能关注点”组织的 Composition API。
3. 渲染层：从更偏通用的运行时更新，走向“编译提示 + 精准 patch”的现代更新模型。
4. 工程层：从“会写 Vue 组件”走向“会写 SFC、TS、组合式逻辑、状态管理、SSR、构建链路”。

真正的区别不只是 API 名字变了，而是 Vue 3 把很多过去“够用但不够优雅”的地方都系统性重做了一遍。

常见坑

1. 只会背 `beforeDestroy -> onBeforeUnmount` 这类表层差异。
2. 把 Vue 2 和 Vue 3 的区别讲成一串 breaking changes 列表。
3. 只讲 Composition API，不讲渲染优化和工程体系升级。

面试展开

比较完整的说法是：Vue 2 和 Vue 3 的差异不能只看 API 变化。Vue 3 在响应式上换成 `Proxy`，在组件组织上补齐了 Composition API，在渲染上通过编译器和运行时协作做精准更新，在工程上把 SFC、TS、SSR、工具链都做得更现代。所以 Vue 3 的升级不是“多了一套写法”，而是整个框架的实现思路都更适合大型项目。

1.6 面试表达

定位

这一节给你一个能直接带去面试的收口，让第一章不只是“看懂”，而是“能讲出来”。

必会结论

1. 讲 Vue，优先讲链路，不要上来背 API。
2. 讲 Vue 3，优先讲系统升级，不要上来只讲 Composition API。
3. 讲性能，优先讲“编译期 + 响应式 + 运行时 patch”的协作，不要只讲某一个点。

核心理解

一个 30 秒版本可以这么说：

Vue 本质上是一套从单文件组件编译、到响应式状态驱动、到运行时渲染更新的完整前端框架。Vue 3 的核心升级不只是 API 更现代，而是响应式从 `defineProperty` 升级到 `Proxy`，模板编译器能提前给运行时提供优化信息，运行时再结合 VNode 和 patch 做更精准的 DOM 更新，所以它的提升是整条链路一起完成的。

一个 90 秒版本可以继续往下展开：

我通常会把 Vue 拆成三层来看。第一层是编译层，SFC 会经过 `compiler-sfc` 和模板编译，最终产出 render 函数；第二层是响应式层，组件在 render 时读取状态，建立依赖，状态变化时触发组件更新；第三层是运行时层，新的 render 结果会生成新 VNode，再通过 patch 更新真实 DOM。Vue 3 比 Vue 2 更强，不只是因为 `Proxy`，还因为编译器会做静态提升、动态标记和子树优化，让运行时不必做无差别比较。

常见坑

1. 上来先背生命周期表。
2. 上来先背 `ref`、`reactive`、`watch` 的语法。
3. 把“Vue 3 为什么快”答成“因为用了 `Proxy`”。

面试展开

如果让我把第一章压成一句话，我会这么答：Vue 不是一堆 API，而是一条从 SFC 编译、到响应式驱动、到运行时 patch 的完整链路；Vue 3 的本质升级，是让这条链路在工程组织、类型系统和更新性能上都变得更现代、更可控。

2 模板、组件与渲染入口

2.1 模板不是字符串替换

定位

这一节先把模板的本质摆正。很多人会写模板，但脑子里仍然把它理解成“把数据塞进 HTML 字符串”，这会直接影响后面对渲染机制的理解。

必会结论

1. Vue 模板不是运行时逐段拼接的字符串模板。
2. 模板通常会在编译阶段被转成 render 函数。
3. 组件更新时，真正参与更新的是 render 产出的 VNode，而不是原始模板文本。

核心理解

模板只是更适合人写的声明式描述方式。开发者看到的是 `<template>`，Vue 真正执行的是编译后的 render 函数。也正因为模板会先进入编译阶段，Vue 才能提前知道哪些节点是静态的，哪些属性是动态的，哪些子树值得重点关注。模板的价值不是“比 JSX 更短”，而是它让编译器更容易做静态分析，进而把优化信息交给运行时。

常见坑

1. 以为模板和运行时是一回事。
2. 以为模板更新就是整段 HTML 重新生成。
3. 只知道模板语法，不知道模板为什么能被优化。

面试展开

如果面试官问模板和 render 的关系，我会说：模板只是 Vue 提供的声明式写法，构建型项目里它通常会先被编译成 render 函数。后续组件渲染和更新，都是 render 生成 VNode，再由 patch 去更新真实 DOM，所以模板不是字符串替换，而是更上层的人类可写语法。

2.2 组件是 Vue 的组织单位

定位

这一节回答“为什么 Vue 一切都围绕组件展开”。只有先把组件理解成组织边界，而不是一个语法盒子，后面的通信、状态和性能才好讲。

必会结论

1. Vue 应用是由组件树组成的。
2. 组件不只是复用 UI 的方式，也是状态边界、依赖边界和更新边界。
3. Vue 的很多机制，包括 props、slots、emit、生命周期、KeepAlive，都是围绕组件边界设计的。

核心理解

组件最重要的价值不是“能复用”，而是“能建立清晰边界”。一个组件通常拥有自己的输入、输出、内部状态和渲染逻辑。父组件通过 props 向下传递数据，子组件通过事件向上反馈变化，插槽让父组件把内容注入到子组件的特定位置。正因为组件是基本组织单位，Vue 才能把更新控制在组件粒度，而不是把页面当成一个无法拆分的大模板。

常见坑

1. 把组件拆分理解成纯粹的 UI 拆分。
2. 组件边界混乱，既改 props 又到处用组件实例 ref 硬控。
3. 组件过大，导致逻辑、状态和模板全堆在一起。

面识展开

更成熟的表述是：在 Vue 里，组件既是 UI 组织单位，也是状态和更新的边界。我们平时讲组件通信，本质上是在讲边界如何协作；我们平时讲组件更新，本质上是在讲哪些状态变化会让哪个边界重新执行 render。

2.3 `setup` 和 `<script setup>` 是现代 Vue 的默认入口

定位

这一节建立现代 Vue 组件的主入口认知，避免把 Vue 3 仍然学成 Vue 2 风格。

必会结论

1. Vue 3 里组件逻辑的核心入口是 `setup()`。
2. `<script setup>` 是 `setup()` 的编译期语法糖，不是另一套运行时机制。
3. 现代 Vue 工程默认应以 Composition API 和 `<script setup>` 为主。

核心理解

`setup()` 的地位很关键，它发生在组件初始化阶段，负责组织组件内部的响应式状态、计算属性、侦听器、组合式逻辑和生命周期注册。`<script setup>` 则是在编译阶段把顶层代码自动放进组件

的 `setup` 语义里，同时减少很多样板代码。它的核心价值不是“写得更短”，而是让逻辑暴露、类型推断和编译期宏都更自然。`defineProps`、`defineEmits`、`defineModel`、`defineExpose` 这些能力，也都建立在这个入口之上。

常见坑

1. 把 `<script setup>` 误解成一个新的运行时 API。
2. 仍然沿用 Vue 2 的按 `data/computed/methods/watch` 分区思维。
3. 进入 Vue 3 之后仍优先写 Options API，只把 Composition API 当补充。

面试展开

可以这样讲：`setup` 是 Vue 3 组件逻辑的核心入口，`<script setup>` 只是它的编译期语法糖。Vue 3 的优势不只是写法更短，而是让逻辑能按功能关注点组织，更适合复用、抽离和类型推断，所以现代工程里我会默认用 `<script setup lang="ts">` 来写组件。

2.4 生命周期要按“实例和 DOM 的时机”来理解

定位

这一节不背表格，而是建立“什么时候能做什么”的时机感。

必会结论

1. 生命周期的关键不是名字，而是组件实例、render 和 DOM 在什么阶段可用。
2. `setup()` 负责初始化逻辑，但这时还不能把它理解成“DOM 已经准备好”。
3. `onMounted`、`onUpdated`、`onUnmounted` 等钩子，本质上是在组件更新链路中的时机节点。

核心理解

理解生命周期最好的方式不是背诵中文名称，而是按时机去记。组件初始化时先处理 props、创建实例、执行 `setup`；随后 render 生成子树并完成挂载，到了 `onMounted` 才适合访问模板 ref 和依赖 DOM 的第三方库；后续状态变化触发更新后，`onUpdated` 才对应“本轮 DOM 已更新”；组件卸载前后则用来清理订阅、定时器和外部实例。KeepAlive 场景下，又会多出激活和失活这条缓存态生命周期。

常见坑

1. 在 `setup` 里直接把 DOM 当成已可用。
2. 在 `onUpdated` 里放高成本逻辑，导致更新越来越慢。
3. 异步分支里再注册生命周期钩子，导致注册时机错误。

面试展开

更稳的表达是：Vue 生命周期最好按组件实例和 DOM 的时机来理解。`setup` 是组件初始化逻辑入口，`onMounted` 对应首次 DOM 可用，`onUpdated` 对应本轮更新后的 DOM 状态，`onUnmounted` 负责清理副作用。这样理解比死记名字更不容易错。

2.5 模板引用是“必要时访问实例或 DOM”的通道

定位

这一节专门处理模板 ref 的定位，避免把它用成组件通信主渠道。

必会结论

1. 模板 ref 主要用于访问 DOM 或少量组件公开能力。
2. 它不是日常状态流和组件协作的首选方案。
3. 在 Vue 里，默认仍然应优先 props、emit、组合式逻辑，而不是父组件拿子组件 ref 硬操作。

核心理解

模板 ref 适合做两类事：第一类是访问真实 DOM，比如输入框聚焦、测量尺寸、初始化图表；第二类是访问子组件显式暴露出的少量方法，比如 `focus()`、`open()`。如果一个页面大量依赖父组件直接调用子组件内部细节，通常说明边界设计出了问题。模板 ref 是逃生舱，不是主数据流。

常见坑

1. 把组件 ref 当成 props/emit 的替代品。
2. ref 还没挂载好就读取，拿到 `null`。
3. 子组件不做显式暴露，父组件却依赖内部实现细节。

面试展开

如果被问到模板 ref，我会说：它在 Vue 里当然重要，但定位很明确，主要用于 DOM 访问和少量 imperative 能力暴露。正常组件协作还是应该走单向数据流，ref 只能作为必要时的补充，而不是默认方案。

2.6 渲染入口最终会落到 render 和 VNode

定位

这一节给模板、组件、生命周期做一次收口，把“我们平时到底是在驱动什么”讲清楚。

必会结论

1. 无论你写模板还是写 render function，组件最终都要产出 VNode。
2. VNode 是运行时描述 UI 的中间结构，不是 Vue 的最终目的。
3. render 的重新执行，是组件更新链路里的关键动作。

核心理解

模板和 render function 只是两种声明 UI 的方式，但它们都会收敛成 VNode。原生元素会生成对应元素节点的 VNode，组件也会生成组件类型的 VNode。运行时识别到组件 VNode 时，会继续创建组件实例并递归处理它的子树。理解这一点后，很多概念会自然串起来：为什么状态变化会触发 render，为什么 render 产出的是 VNode，为什么后面还需要 patch，为什么组件更新本质上是“重新描述一次 UI，再把差异落地”。

常见坑

1. 把模板、render、VNode、DOM 混成同一层概念。
2. 以为组件更新就是直接改 DOM。
3. 不了解 render 在更新链路中的位置，只会从模板表面理解 Vue。

面试展开

我一般会这样收口第二章：Vue 的模板、组件和生命周期，本质上都在服务于渲染入口。开发者写的是组件和模板，运行时执行的是 render，render 生成的是 VNode，最后再通过 patch 同步到 DOM。把这条线看清楚，后面的响应式、diff、性能优化就不会散。

3 响应式与状态建模

3.1 响应式不是魔法，是依赖追踪

定位

这一节先把响应式的本质讲清楚，不急着列 API。

必会结论

1. 响应式的核心是读取时收集依赖，修改时触发副作用。
2. Vue 不会“自动知道一切变化的意义”，它只是根据访问和修改建立依赖图。
3. 组件渲染本身也是响应式系统驱动的一个副作用。

核心理解

所谓响应式，并不是数据自己会动，而是 Vue 在你读取状态时记住“谁依赖了它”，在你修改状态时通知“依赖它的逻辑需要重新执行”。组件 render、计算属性求值、侦听器回调，底层都可以放进这套模型里理解。也正因为如此，响应式不是 Vue 的附属功能，而是 Vue 更新链路的发动机。

常见坑

1. 把响应式理解成“只要变量变了页面就会变”。
2. 不知道依赖是“读取时”建立的，导致很多行为判断错位。
3. 只会用 API，不知道为什么解构、浅响应、逃生舱会影响结果。

面试展开

更好的答法是：Vue 响应式本质上是一个依赖图模型。读取状态时做依赖收集，修改状态时触发关联副作用。组件 render effect、computed、watch 都只是这套机制上不同层次的封装。

3.2 `ref` 和 `reactive` 是两种状态建模思路

定位

这一节不只是教语法，而是讲状态怎么建模更稳。

必会结论

1. `ref` 更适合值语义明确、可替换、易传递的状态。
2. `reactive` 更适合对象容器语义明确的状态。
3. 在现代 Vue 工程里，很多团队会优先用 `ref`，再在需要对象容器时使用 `reactive`。

核心理解

`ref` 的核心价值是显式。它把“这是一个响应式值”这件事表达得很清楚，重赋值、跨函数传递和 TypeScript 推断通常也更稳定。`reactive` 的核心价值是对象语义自然，适合一组强相关字段共同构成状态容器的场景。但 `reactive` 也会带来代理身份、解构丢失响应性等问题，所以它更适合“我明确需要一个整体对象”的情况，而不是见对象就包一层。

常见坑

1. 误以为 `reactive` 比 `ref` 更高级。
2. 所有状态都堆进一个巨大的 `reactive` 对象里。
3. 在脚本里忘记 `ref` 需要通过 `.value` 访问。

面试展开

如果被问 `ref` 和 `reactive` 怎么选，我会说：我更倾向先用 `ref` 建模单值和明确边界的状态，因为它更显式、重赋值更自然、TS 推断也更稳；`reactive` 则用于对象容器或一组强耦合字段的整体语义。不是谁更高级，而是谁更符合状态形状。

3.3 解构会改变响应式边界

定位

这一节处理 Vue 响应式里最经典也最容易面试追问的点。

必会结论

1. `reactive` 解构后会丢失原本的响应式连接。
2. `toRef` 适合单字段保持连接，`toRefs` 适合整体安全解构。
3. 在 props 或 store 场景里，这个问题尤其高频。

核心理解

`reactive` 的响应性来自代理对象本身，而不是解构出来的值。你一旦把某个字段直接解出来，后续对这个变量的访问就不再经过原代理对象，自然也就失去了依赖追踪。`toRef` 和 `toRefs` 的价值，就是把这些字段重新包装成保留连接关系的 `ref`。这个心智不只适用于普通状态，也适用于 props、Pinia store 等一切基于代理对象的场景。

常见坑

1. 直接解构 `reactive` 或 store，然后以为它还是响应式的。
2. props 传进 composable 之后失去更新。
3. 不区分“编译期响应式 props 解构”和普通运行时解构的边界。

面试展开

更成熟的表达是：`reactive` 的依赖收集发生在你通过代理对象读取属性的那一刻，直接解构会绕开这个过程，所以响应性会断掉。`toRef` 和 `toRefs` 的本质，就是把字段重新接回响应式系统。

3.4 `computed` 代表纯派生状态

定位

这一节把 `computed` 从“会算值的 API”提升到状态建模原则。

必会结论

1. `computed` 适合描述纯派生状态，不适合承载副作用。
2. `computed` 的核心优势是惰性求值、缓存和依赖失效后重算。
3. 能用 `computed` 表达的，就不要用 `watch` 去手动维护一个派生值。

核心理解

`computed` 最核心的价值不是语法简洁，而是让“派生关系”变成声明式结构。只要源状态不变，计算结果就直接复用；只有依赖变化，它才会重新求值。这让状态流更稳定，也能减少很多多余计

算。工程里最常见的误区，是明明只是“由 A、B 推出 C”，却非要用 `watch` 去手动改 `C`，结果把派生状态写成了命令式同步逻辑。

常见坑

1. 在 `computed` 里发请求、改状态、改 DOM。
2. 用 `watch` 人工维护可由 `computed` 推导出的数据。
3. 把复杂业务流程硬塞进单个 `computed`，导致难以维护。

面试展开

如果我被问 `computed` 的定位，我会说：它不是简化版函数，而是 Vue 对“派生状态”的建模工具。只要这个值本质上由其他响应式状态推导出来，而且应该保持纯净和可缓存，那就优先用 `computed`。

3.5 深响应不是默认越多越好

定位

这一节开始引入工程语境里的性能边界。

必会结论

1. 深响应很方便，但不是所有数据都值得被深度代理。
2. 大对象、第三方实例、只关心整体替换的数据，通常更适合浅响应或跳过代理。
3. Vue 3 提供 `shallowRef`、`shallowReactive`、`markRaw` 这些工具，就是为了管理这类边界。

核心理解

Vue 3 的深响应默认体验很好，但工程里不能把“默认方便”误解成“全量最优”。有些数据结构很大，有些对象来自图表、地图、编辑器等第三方库，有些状态我们只关心整体替换而不关心内部细节。这时如果仍然做深度代理，往往既增加理解成本，也增加不必要的响应式开销。浅响应和 `markRaw` 的价值，不是炫技巧，而是精确表达“哪些值得追踪，哪些不值得”。

常见坑

1. 把编辑器实例、图表实例直接塞进深响应对象。
2. 面对大型只读结构仍然默认深度代理。
3. 不理解浅响应和深响应的边界，导致更新判断混乱。

面试展开

更工程化的回答是：Vue 响应式不是越深越好。对大对象、第三方实例或只关心整体替换的数据，我会优先考虑 `shallowRef`、`shallowReactive` 或 `markRaw`。响应式的目标是提高可维护性和更新效率，不是把一切都代理化。

3.6 `toRaw` 和逃生舱是边界工具，不是默认手段

定位

这一节把响应式系统的“例外通道”讲清楚。

必会结论

1. `toRaw`、`markRaw` 这类能力属于逃生舱，不是日常默认写法。
2. 它们的价值在于兼容第三方库、调试底层行为、规避不必要追踪。
3. 一旦使用逃生舱，就要更清楚地管理状态边界和身份语义。

核心理解

响应式系统的设计目标是让大多数数据流足够自然，但工程里总有少量特殊情况需要绕开默认轨道。比如某些第三方实例不适合被代理，某些调试或性能场景需要拿到原始对象，这时 `toRaw` 和 `markRaw` 就很有价值。但必须记住，它们不是“性能优化万能钥匙”，更像是明确的 escape hatch。用得越多，说明状态边界越需要重新设计。

常见坑

1. 把 `toRaw` 当成常规优化工具长期持有。
2. 用了原始对象后又忘记它不再参与正常追踪。
3. 为了省事大量逃离响应式系统，最后把代码写成半响应式半手动维护。

面试展开

如果面试官问 `toRaw` 的定位，我会说：它是响应式系统的逃生舱，适合少量底层兼容和调试场景，不适合日常建模。真正健康的工程状态流，还是应尽量保持在 Vue 的正常响应式语义里。

3.7 状态建模的工程原则

定位

这一节把前面的 API 收成一套可落地的建模习惯。

必会结论

1. 状态建模优先追求清晰边界，其次才是语法偏好。
2. 区分源状态、派生状态、副作用，是 Vue 可维护性的关键。

3. Vue 里的很多问题，表面看是 API 用错，根上其实是状态建模混乱。

核心理解

一个更稳的 Vue 状态模型，通常会遵守三条原则。第一，源状态尽量少而明确，只保留真正需要被持有的数据。第二，派生状态交给 `computed`，不要重复存储。第三，副作用交给侦听器和生命周期，而不是让 `computed` 或模板表达式偷偷做事。只要这三层分清，响应式系统就会更可预测，组件也会更容易拆分、测试和优化。

常见坑

1. 同一份信息既做源状态又做派生状态，造成双写。
2. 派生逻辑、副作用逻辑、渲染逻辑混在一起。
3. 先堆 API，再想边界，导致组件越来越乱。

面试展开

如果让我总结 Vue 状态建模，我会说：关键不是记住多少响应式 API，而是先分清哪些是源状态，哪些是派生状态，哪些是副作用。`ref/reactive` 解决的是“状态怎么持有”，`computed` 解决的是“状态怎么推导”，后面的 `watch/watchEffect` 解决的是“变化后要做什么事”。只要这三层分清，Vue 代码就会稳定很多。

4 侦听、副作用与逃生舱

4.1 `watch`、`watchEffect`、`computed` 先分工再写代码

定位

这一节先把三者边界压清楚。很多 Vue 代码难维护，不是 API 不会用，而是派生状态和副作用混着写了。

必会结论

1. `computed` 负责纯派生状态。
2. `watch` 负责监听明确来源后的副作用。
3. `watchEffect` 负责“先执行副作用，再自动收集同步阶段依赖”。

核心理解

一个很稳的判断方式是：如果你在描述“这个值由哪些状态推导出来”，优先 `computed`；如果你在描述“某个变化发生后我要做一件事”，优先 `watch`；如果你只关心“这个副作用里读到了什么，就跟着什么变化”，再用 `watchEffect`。很多人把 `watch` 当万能胶，结果把派生值也写成命令式同步逻辑，状态流就会越来越乱。

常见坑

1. 用 `watch` 维护本可由 `computed` 推导出的状态。
2. 在 `computed` 里发请求、改状态、改 DOM。
3. 不区分派生状态和副作用，导致代码越来越命令式。

面试展开

我通常会先把三者关系讲成一句话：`computed` 解决“值怎么推导”，`watch` 解决“变化后做什么”，`watchEffect` 解决“自动依赖的副作用怎么跑”。只要这层分工清楚，后面的侦听器设计就很稳。

4.2 `watch` 适合明确依赖和可控副作用

定位

这一节讲 `watch` 的真正强项，不只是语法。

必会结论

1. `watch` 适合你明确知道要监听谁的场景。
2. `watch` 能拿到新值、旧值，并能精确控制触发时机。
3. 路由参数变化、表单联动、异步请求、缓存同步，这些都更适合 `watch`。

代码抓手

代码块

```
1  import { ref, watch } from 'vue'
2  const keyword = ref('')
3  const result = ref<string[]>([])
4  watch(keyword, async (val) => {
5    if (!val.trim()) {
6      result.value = []
7      return
8    }
9    result.value = await searchApi(val)
10 }, { immediate: true })
```

核心理解

`watch` 的价值在于显式和可控。你知道依赖源是谁，也知道回调什么时候执行，还能拿到旧值和清理时机。这让它非常适合异步请求、路由变化、外部同步、DOM 联动等副作用场景。它不是比 `watchEffect` 更高级，而是更适合那些依赖明确、需要控制边界的逻辑。

常见坑

1. 直接监听整个大对象，结果深层追踪成本过高。
2. 监听 `reactive` 某个属性时不用 `getter`，导致写法不稳。
3. 把多个本不相关的副作用都塞进同一个 `watch`。

面试展开

更工程化的回答是：`watch` 最适合那些我明确知道依赖源、需要拿到新旧值、还要控制触发节奏和清理逻辑的场景，比如路由参数驱动的数据请求、状态同步、本地缓存更新、滚动或标题联动。

4.3 `watchEffect` 适合自动依赖副作用

定位

这一节处理 `watchEffect` 的正确定位，避免把它神化。

必会结论

1. `watchEffect` 会立即执行。
2. 它会自动收集同步执行阶段访问到的响应式依赖。
3. 它更适合依赖源分散、但副作用本身很直接的场景。

代码抓手

代码块

```
1 import { ref, watchEffect } from 'vue'
2 const count = ref(0)
3 watchEffect(() => {
4   document.title = `当前计数: ${count.value}`
5 })
```

核心理解

`watchEffect` 的优势是省心。你不用先维护一个依赖源列表，只要在副作用函数里同步访问到了哪些响应式状态，这些状态变化后它就会重新执行。这很适合标题同步、简单日志、副作用联动、轻量数据拼接这类场景。但它不是万能方案，因为它拿不到旧值，也不适合依赖边界特别复杂的逻辑。

常见坑

1. 误以为 `watchEffect` 会自动追踪 `await` 之后的依赖。
2. 把非常复杂的异步流程塞进 `watchEffect`，结果依赖来源难定位。

3. 用它替代所有 `watch`，最后回调里谁触发了谁都讲不清。

面试展开

如果面试官问 `watchEffect` 的边界，我会说：它适合“副作用和依赖天然写在一起”的场景，但只会追踪同步阶段访问到的依赖。只要我需要明确依赖源、旧值、清理时机或更强控制，我还是会回到 `watch`。

4.4 侦听器不是只会“监听”，还要会“清理”

定位

这一节把侦听器从“回调触发器”升级成“可管理生命周期的副作用容器”。

必会结论

1. 侦听器里最容易出问题的不是触发，而是清理。
2. 异步请求、事件订阅、定时器、副作用竞争，都必须考虑失效清理。
3. Vue 3.5+ 的 `onWatcherCleanup()` 很适合处理侦听器竞态。

代码抓手

代码块

```
1 import { ref, watch, onWatcherCleanup } from 'vue'
2 const id = ref(1)
3 watch(id, async (newId) => {
4   const controller = new AbortController()
5   onWatcherCleanup(() => controller.abort())
6   const res = await fetch(`/api/detail/${newId}`, {
7     signal: controller.signal
8   })
9   console.log(await res.json())
10 })
```

核心理解

真正的难点不是“值变化后执行了”，而是“上一次副作用还没结束怎么办”。最典型的是竞态请求：用户快速切换 id，如果你不清理旧请求，慢返回的旧结果可能覆盖新结果。侦听器一旦承载异步副作用，就一定要把清理当默认动作。`onWatcherCleanup()` 的价值，就是把“本轮副作用失效时要做什么”明确绑定到当前这次执行里。

常见坑

1. 连续切换条件时，旧请求回来了把新数据覆盖掉。

2. 侦听器里注册了事件和定时器，却没有回收。
3. 在 `await` 之后再调用 `onWatcherCleanup()`，错过同步注册时机。

面试展开

我一般会强调：侦听器真正考验工程能力的地方在清理，而不在监听本身。尤其是请求、订阅和资源占用类副作用，如果没有和当前这次侦听执行绑定清理逻辑，就容易出现竞态和泄漏。

4.5 触发时机要有意识： `immediate`、`once`、`flush`

定位

这一节把 `watch` 的时机控制讲清楚，不然很多“为什么不对”的问题都解释不明白。

必会结论

1. `immediate` 让侦听器在初始化时先执行一次。
2. `once` 适合只需要响应首次变化的场景。
3. `flush` 决定副作用在更新队列中的执行时机。

代码抓手

代码块

```
1  watch(source, callback, {
2    immediate: true,
3    once: false,
4    flush: 'post'
5  })
```

核心理解

时机控制的关键不是记参数名，而是知道你想拿到“变化前的什么”还是“变化后的什么”。如果你需要初始化就拉一次数据，用 `immediate`；如果你只关心第一次变化，用 `once`；如果你要拿更新后的 DOM，就用 `flush: 'post'`。默认时机通常够用，但一旦碰到 DOM 测量、动画联动、第三方实例同步，就必须对 `flush` 有明确认识。

常见坑

1. 需要更新后的 DOM，却在默认时机里直接测量。
2. 重复写“先执行一次函数，再写一个 `watch`”，本质上其实是 `immediate` 场景。
3. 为了追求“立刻执行”滥用 `flush: 'sync'`。

面试展开

如果被问侦听器的触发时机，我会说：最常用的是 `immediate` 和 `flush`。前者解决初始化是否需要先跑一次，后者解决副作用到底应挂在更新前还是更新后。DOM 相关逻辑通常更适合 `flush: 'post'`。

4.6 深监听要慎用，数组监听也有版本语义

定位

这一节专门处理高频误区：很多 Vue 性能问题都不是“框架不行”，而是深监听滥用。

必会结论

1. 深监听成本高，尤其是在大型对象和复杂数组上。
2. 监听 `reactive` 整体对象时，深层变化经常会让 `newValue` 和 `oldValue` 指向同一引用。
3. Vue 3.5+ 里 `deep` 可以是数字，用来限制遍历深度。

核心理解

深监听的本质，是让 Vue 去递归遍历对象结构并建立更多追踪关系。这当然方便，但并不免费。很多时候你真正关心的只是某个字段、某类变化、某个派生结果，而不是整个状态树。此时更稳的方式通常是缩小监听源，而不是直接对整个大对象开深监听。数组也是同理，监听“整体替换”和监听“内部变异”不是一回事。

常见坑

1. 对全量表单对象、全量 schema、全量 store 直接深监听。
2. 看到 `oldValue` 和 `newValue` 一样就以为 watch 失效了。
3. 不区分“监听数组引用变化”和“监听数组内部变异”。

面试展开

更成熟的说法是：深监听是能力，不是默认策略。对象越大、结构越深、引用越复杂，就越应该把监听源收窄。真正稳定的工程代码，通常不是“监听一切”，而是“监听最小必要变化”。

4.7 `nextTick` 解决的是“等本轮 DOM 更新完成”

定位

这一节把 `nextTick` 从常用小技巧上升到调度认知。

必会结论

1. Vue 的状态修改不是每改一次就同步 patch 一次 DOM。
2. Vue 会把同一轮里的更新批量调度。
3. `nextTick` 的价值是等待本轮队列刷新后的 DOM 状态。

代码抓手

代码块

```
1 import { ref, nextTick } from 'vue'
2 const visible = ref(false)
3 async function open() {
4   visible.value = true
5   await nextTick()
6   document.querySelector('#panel')?.scrollIntoView()
7 }
```

核心理解

很多人把 `nextTick` 理解成“延迟一下”，其实更准确的说法是“等 Vue 完成这一轮更新刷新”。Vue 不会每次状态变化都立刻改 DOM，而是会把更新任务入队、去重、批量刷新。也正因为如此，当你在状态修改后立刻访问 DOM，拿到的可能还是旧状态。`nextTick` 解决的不是异步业务问题，而是渲染调度时机问题。

常见坑

1. 把 `nextTick` 当成通用异步工具到处乱包。
2. 不需要 DOM 后状态时也机械使用 `nextTick`。
3. 不理解它和调度队列的关系，只把它当“写着保险”。

面试展开

我会这样讲：`nextTick` 不是为了“慢一点”，而是为了“等这一轮 Vue 更新真正落到 DOM”。它的使用场景通常是状态改完后立刻要测量、聚焦、滚动或驱动第三方 DOM 库。

4.8 `effectScope` 是组合式副作用的回收边界

定位

这一节开始把副作用管理提升到 composable 和 store 级别。

必会结论

1. `effectScope()` 可以把一组响应式副作用收拢到同一个作用域。

2. 作用域 stop 后，内部的 watch、watchEffect、computed 等都能一起停止。
3. 这在 composable、插件、Pinia、临时模块里非常有价值。

代码抓手

代码块

```
1 import { effectScope, ref, watch } from 'vue'
2 const scope = effectScope()
3 scope.run(() => {
4   const count = ref(0)
5   watch(count, () => {
6     console.log(count.value)
7   })
8 })
9 scope.stop()
```

核心理解

组件天然有生命周期，但 composable、插件和某些共享逻辑不一定有现成的组件边界。这时如果内部又创建了很多侦听器 and 派生状态，就需要一个统一的回收容器。`effectScope` 的价值，就是给这些脱离单个组件的响应式逻辑提供统一生命周期管理能力。

常见坑

1. composable 内部创建了很多侦听器，却没有统一释放边界。
2. 误以为只有组件卸载才能停止副作用。
3. 用完 `effectScope` 却忘了在合适时机 stop。

面试展开

如果我想把这一点讲得更工程一点，会说：`effectScope` 解决的是“组件之外的响应式副作用怎么成组管理”。在大型应用和库设计里，这是非常有价值的能力。

4.9 `shallowRef`、`markRaw`、`toRaw` 是逃生舱，不是主航道

定位

这一节把前面提到的响应式边界工具正式收束。

必会结论

1. `shallowRef` 适合只关心整体替换、不关心内部深层追踪的大对象。
2. `markRaw` 适合彻底跳过代理的第三方实例或特殊对象。

3. `toRaw` 适合少量调试、兼容和逃逸场景，不适合长期持有。

代码抓手

代码块

```
1 import { shallowRef, markRaw } from 'vue'
2 const chart = shallowRef<any>(null)
3 function initChart(instance: any) {
4   chart.value = markRaw(instance)
5 }
```

核心理解

这些 API 的共同点是：都在帮助你明确响应式边界。真正成熟的 Vue 工程不是把一切都交给深响应，而是知道哪里该深、哪里该浅、哪里该完全跳过。这些能力很重要，但它们属于例外处理，不是默认建模方式。

常见坑

1. 为了“优化”到处用 `toRaw`，反而把响应式语义打碎。
2. 用 `markRaw` 后忘了对象内部不再受 Vue 正常追踪。
3. 把浅响应误解成“性能更高所以应该默认使用”。

面试展开

我一般会强调：这些 API 的价值不在于“更高级”，而在于明确边界。大对象、编辑器实例、图表实例、地图实例，往往不该无脑深代理，这时浅响应和跳过代理才真正有意义。

5 组件通信与状态边界

5.1 默认先守住单向数据流

定位

这一章先把组件通信的总原则立起来，不然后面的 `props`、`emit`、`v-model`、`provide/inject` 都会被误用。

必会结论

1. Vue 组件通信的默认模型仍然是单向数据流。
2. 父组件负责传入状态，子组件负责消费状态和抛出事件。
3. 子组件直接改 props，是最基础也最危险的边界破坏。

核心理解

Vue 虽然提供了很多通信方式，但默认最稳的仍然是“父传子，子通知父，父来改状态”。这让数据来源清晰、更新路径可追踪、问题更容易定位。很多复杂度看起来像是业务复杂，其实是因为边界一开始就没收住，结果组件之间互相改状态、互相拿 ref、互相耦合。

常见坑

1. 子组件直接改 props。
2. 父组件通过 ref 深度操控子组件内部实现。
3. 通信通道太多，导致状态来源不清楚。

面试展开

如果让我概括组件通信，我会先说一句：Vue 所有通信能力里，默认最稳的是单向数据流。先用 `props + emit` 解决，只有在层级过深或语义不同的时候，再考虑 `provide/inject`、`store` 或其他手段。

5.2 `defineProps` 和 `defineEmits` 是组件契约

定位

这一节把 props 和 emits 从“语法糖”讲成“公开接口”。

必会结论

1. `props` 定义的是组件输入契约。
2. `emits` 定义的是组件输出契约。
3. 组件是否好维护，很大程度取决于输入输出边界是否清楚。

代码抓手

代码块

```
1  <script setup lang="ts">
2  const props = defineProps({
3    title: string
4    disabled?: boolean
5  })
6  const emit = defineEmits({
7    submit: [value: string]
8  })
9  function handleSubmit() {
10    emit('submit', props.title)
11  }
```

核心理解

成熟的组件设计不是“能用就行”，而是输入输出要稳定。`defineProps` 让你明确组件需要什么信息，`defineEmits` 让你明确组件会把什么变化抛给外部。它们不是单纯为了类型舒服，而是帮助团队把组件当成一个可推理的边界来维护。

常见坑

1. 组件输入太多，职责已经开始失控。
2. 事件命名和语义混乱，父组件根本看不出触发意图。
3. 组件一边收 props，一边又在内部偷偷维护一份镜像状态。

面试展开

我会把 `props/emits` 讲成组件协议。一个组件如果输入不清晰、输出不稳定，后面无论是测试、复用还是性能优化都会很痛苦。

5.3 `v-model` 是受控能力，不是双向绑定泛滥

定位

这一节把 `v-model` 拉回工程语义，避免它被理解成“方便就行”。

必会结论

1. 组件级 `v-model` 本质上仍然是 `prop + emit` 协议。
2. Vue 3.4+ 的 `defineModel()` 只是把样板代码藏到了编译期。
3. `v-model` 适合输入类、开关类、受控值类组件，不适合泛化成所有业务通信方式。

代码抓手

代码块

```
1  <script setup lang="ts">
2  const model = defineModel<string>()
3  </script>
4  <template>
5    <input v-model="model" />
6  </template>
```

核心理解

`v-model` 看起来像双向绑定，但在组件语义上，它本质上仍然是单向数据流上的一个受控接口约定。父组件提供值，子组件在内部通过事件请求更新。`defineModel()` 的价值是减少模板代码，不是改变数据流本质。也正因为如此，`v-model` 更适合封装受控输入能力，而不是把所有父子协作都写成“互相同步”。

常见坑

1. 看到 `defineModel()` 就拿来替代所有 `props + emits`。
2. 子组件有 `default` 值，父组件却没传，导致内外状态脱节。
3. 一般业务组件滥用 `v-model`，反而让状态路径变模糊。

面试展开

更稳的说法是：`defineModel` 是 Vue 3.4+ 提供的编译期宏，用来简化组件级 `v-model` 实现。本质上它仍然是 `modelValue + update:modelValue` 这套协议，所以它适合封装受控组件，但普通业务协作我还是更倾向明确的 `props + emits`。

5.4 插槽是内容分发，不是数据反向穿透

定位

这一节处理 slots 的真实定位，避免把它当万能注入口。

必会结论

1. 插槽解决的是父组件向子组件注入内容结构的问题。
2. 作用域插槽解决的是子组件向父组件暴露渲染所需数据的问题。
3. 插槽内容的作用域仍属于父组件，不属于子组件。

核心理解

普通 props 适合传数据，插槽适合传结构。一个组件想把“外壳和能力”固定住，同时允许调用方控制局部渲染，就非常适合 slots。作用域插槽则进一步让子组件把一部分上下文暴露给父组件，让父组件决定怎么渲染。真正需要记住的是：插槽的本质是内容分发和渲染协作，不是状态管理替代品。

常见坑

1. 把插槽当成跨层状态共享通道。
2. 不理解插槽作用域归属，结果变量来源一团乱。
3. 能用简单 props 的地方硬上复杂作用域插槽。

面试展开

如果面试官问 slots，我会说：它解决的不是“怎么传数据”，而是“怎么把渲染控制权合理交出去”。props 更适合数据输入，slots 更适合内容注入和可组合 UI。

5.5 provide/inject 解决的是深层依赖，不是偷懒通信

定位

这一节把 provide/inject 放回正确位置，不让它和全局状态管理混淆。

必会结论

1. provide/inject 适合解决深层组件树的依赖传递问题。
2. 它改变的是依赖注入方式，不等于全局状态管理。
3. 大型工程里更推荐 Symbol + InjectionKey<T>，而不是字符串 key。

代码抓手

代码块

```
1 // keys.ts
2 import type { InjectionKey, Ref } from 'vue'
3 export const CountKey = Symbol() as InjectionKey<Ref<number>>
```

代码块

```
1 <script setup lang="ts">
2   import { ref, provide, inject } from 'vue'
3   import { CountKey } from './keys'
4   const count = ref(0)
5   provide(CountKey, count)
6   const injected = inject(CountKey)
7 </script>
```

核心理解

provide/inject 的本质是依赖注入。它很适合主题、表单上下文、列表上下文、组件库内部状态、插件能力这些“祖先组件提供能力，后代组件按需消费”的场景。它解决的是 prop drilling，不是所有共享状态都该走这条路。尤其在大型项目里，如果 key 不做类型和命名隔离，很容易变成隐式耦合。

常见坑

1. 字符串 key 到处复用，后期很难维护。

2. 下层组件拿到共享对象后到处直接改。
3. 用 `provide/inject` 代替清晰的组件接口设计。

面试展开

更工程化的回答是：`provide/inject` 更像局部依赖注入，而不是全局 store。它适合解决跨多层组件共享上下文的问题，但状态修改入口最好仍然集中在 provider 一侧，下层尽量消费而不是随意改写。

5.6 组件实例 ref 是必要时的命令式通道

定位

这一节把“父调子”这类 imperative 协作收边。

必会结论

1. 组件实例 ref 适合暴露少量 imperative 能力。
2. 这种能力应该显式暴露，而不是默认穿透内部实现。
3. 正常状态协作仍然应优先声明式数据流。

核心理解

有些能力天然是指令式的，比如弹窗 `open()`、输入框 `focus()`、重置 `reset()`。这种时候组件实例 ref 很合理。但如果一个父组件需要频繁调用子组件内部很多细节，那往往说明这个组件边界已经不健康了。命令式通道可以有，但要小而明确。

常见坑

1. 父组件对多个子组件内部方法高度依赖。
2. 不做显式暴露，直接耦合子组件内部细节。
3. 能靠状态表达的逻辑也硬写成 imperative 调用。

面试展开

我会这样说：组件 ref 不是不能用，但定位要明确。它更适合少量 imperative 能力暴露，比如 `focus`、`open`、`scrollTo`，而不是取代正常的组件通信模型。

5.7 组件通信的工程判断顺序

定位

这一节把通信手段收束成一套决策顺序。

必会结论

1. 先问是不是单向数据流能解决。
2. 再问是不是深层上下文共享。
3. 再问是不是局部命令式能力。
4. 最后才考虑全局 store 或更高层架构调整。

核心理解

很多组件通信混乱，不是因为手段不够，而是没有判断顺序。最稳的顺序通常是：父子协作先用 `props/emits`，受控值类组件再考虑 `v-model`，深层上下文共享才用 `provide/inject`，只有少量 imperative 能力才用模板 ref。这个顺序一旦建立，很多本来看起来复杂的通信问题都会简单很多。

常见坑

1. 一上来就想“有没有更高级的通信方式”。
2. 还没搞清边界，就直接上全局 store。
3. 在多个通信通道之间来回混搭，最后状态流没人能讲清。

面试展开

如果让我总结组件通信，我会说：通信方式不是越高级越好，而是越匹配边界越好。先守住单向数据流，再按需要向下引入上下文注入、受控组件模式和少量命令式能力。

6 Router、Pinia 与应用组织

6.1 Router 和 Pinia 不是“插件”，而是应用骨架

定位

这一节先把第六章的层级拉高，避免把路由和状态管理讲成 API 教程。

必会结论

1. Router 决定页面组织、导航时机和部分数据获取节奏。
2. Pinia 决定跨页面状态的持有、派生和动作组织。
3. 真正的 Vue 应用组织，离不开路由边界、状态边界和数据边界的协同。

核心理解

一个中大型 Vue 应用不会只是很多组件堆起来，而是要有页面边界、共享状态边界、导航边界和请求边界。Vue Router 负责把 URL、页面、导航、守卫这些关系串起来，Pinia 负责把跨组件和跨页面共享的状态收敛起来。组件树只是表现层，Router 和 Pinia 才是应用层骨架的一部分。

常见坑

1. 把一切页面逻辑都塞进组件树，不做路由层抽象。
2. 把所有状态都塞进 Pinia，结果 store 变成大杂烩。
3. 页面、路由、store、请求没有清晰边界。

面试展开

我会把这一层讲成：Vue 组件解决的是局部 UI 组织，Router 和 Pinia 解决的是应用级组织。真正的大型前端项目，必须把页面边界和共享状态边界拆清楚。

6.2 Vue Router 不只是跳转工具

定位

这一节把 Router 从“路径映射器”提升到应用导航系统。

必会结论

1. Vue Router 的价值不只是页面跳转。
2. 它还负责嵌套路由、导航守卫、路由元信息、滚动行为、懒加载和数据获取协同。
3. 路由层往往是权限、布局、过渡、预取数据的关键入口。

核心理解

Vue Router 真正重要的地方不在“会不会 `push`”，而在它把 URL、页面结构、导航流程和页面级能力统一起来了。比如嵌套路由决定布局嵌套关系，导航守卫决定权限拦截和预加载时机，meta 决定页面级策略，懒加载决定首屏拆包粒度。一个项目的“页面如何组织”，本质上就是 Router 设计问题。

常见坑

1. 只会配置 path 和 component，不会设计路由层职责。
2. 权限、布局、数据获取都散在组件里，路由层没有承担组织责任。
3. 所有路由组件全量同步导入，导致首屏包过大。

面试展开

比较稳的说法是：Vue Router 是应用导航系统，不只是地址变化工具。它既管理页面结构，也管理导航过程中的权限、懒加载、滚动、过渡和部分数据准备逻辑。

6.3 路由组件应优先懒加载

定位

这一节把工程里最有性价比的 Router 优化先讲出来。

必会结论

1. 路由组件默认就应该优先动态导入。
2. 路由懒加载能天然形成代码分割。
3. 页面级代码拆包，通常比细碎组件级拆包更有收益。

代码抓手

代码块

```
1  import { createRouter, createWebHistory } from 'vue-router'
2  const router = createRouter({
3    history: createWebHistory(),
4    routes: [
5      {
6        path: '/users',
7        component: () => import('./pages/users/index.vue')
8      }
9    ]
10  })
```

核心理解

页面路由天然就是拆包边界，因为用户不可能在首屏立刻访问所有页面。把路由组件做成动态导入，能让构建工具直接按页面分 chunk，这对加载性能非常友好。相比之下，过早把每个小组件都异步化，收益反而不一定大，还会带来更多 loading 管理复杂度。

常见坑

1. 路由全同步导入，首屏包臃肿。
2. 过度细粒度拆包，结果请求过多、心智更复杂。
3. 页面主内容还没稳住就到处插异步边界。

面试展开

如果被问路由懒加载，我会说：它通常是 Vue 应用最基础也最有效的代码分割策略，因为页面边界天然就是用户访问边界。相比随意拆小组件，路由级懒加载的收益更直接。

6.4 路由数据获取要看导航时机

定位

这一节讲“数据该在组件里取，还是在导航阶段取”的判断。

必会结论

1. 页面数据获取不是只有 `onMounted` 这一条路。
2. 有些数据适合进入页面后在组件中获取，有些更适合和导航流程绑定。
3. 路由层数据获取的关键是时机一致性，而不是“写在哪儿更顺手”。

核心理解

很多 Vue 项目一谈数据获取就只会在页面组件里 `watch(route.params.id)` 然后发请求，这当然可行，但并不是唯一方案。工程里要考虑的是：这个数据是否会影响页面能否进入、是否需要和导航统一协调、是否要 SSR、是否要避免组件层层挂 loading。路由层和组件层都可以取数，关键是按导航语义决定，而不是凭习惯写。

代码抓手

代码块

```
1 import { useRoute } from 'vue-router'
2 import { ref, watch } from 'vue'
3 const route = useRoute()
4 const detail = ref(null)
5 watch(() => route.params.id, async (id) => {
6   detail.value = await fetchDetail(String(id))
7 }, { immediate: true })
```

常见坑

1. 一切数据都塞进 `onMounted`，导致参数变化时逻辑补丁越来越多。
2. 页面切换、权限拦截、数据依赖没有统一节奏。
3. 只会“组件进来再加载”，不会设计导航前后的策略。

面试展开

更成熟的回答是：页面数据获取要结合导航时机设计。轻量页面内数据可以在组件层获取，但如果数据本身影响页面进入条件、SSR、统一 loading 或导航一致性，就应该考虑和路由流程更紧密地结合。

6.5 Pinia 是 Vue 3 时代更自然的 store 形态

定位

这一节把 Pinia 讲成 Vue 3 响应式模型的自然延伸。

必会结论

1. Pinia 本质上仍然建立在 Vue 响应式系统之上。
2. 它更贴近 Composition API 的思维方式。
3. Pinia 不是“更轻的 Vuex”这么简单，而是更同构于 Vue 3。

核心理解

Pinia 的优势不只是 API 看起来更短，而是它的建模方式和 Vue 3 的响应式、组合式逻辑、TS 推断都更一致。state、getters、actions 不再是那种模板化的强仪式结构，而更像一个可组合的状态模块。也正因为如此，Pinia 在现代 Vue 工程里不只是“官方推荐”，而是思维模型上更顺滑。

常见坑

1. 把 Pinia 只理解成“替代 Vuex 的新库”。
2. store 里什么都放，结果职责越来越重。
3. 组件局部状态也想丢进 Pinia，导致全局状态泛滥。

面试展开

我会这样表达：Pinia 更符合 Vue 3，不只是因为更简洁，而是因为它和 Composition API、响应式系统、TS 推断天然更一致，状态模块的组织方式也更接近现代工程实践。

6.6 `storeToRefs()` 解决的是 store 解构丢失响应性

定位

这一节专门处理 Pinia 最常见的面试点和实际坑点。

必会结论

1. Pinia store 解构后同样会面临响应性丢失问题。
2. `storeToRefs()` 的作用就是把 state、getters 等重新接成 refs。
3. actions 不需要 `storeToRefs()`，它们直接从 store 取即可。

代码抓手

代码块

```
1 import { storeToRefs } from 'pinia'
2 const userStore = useUserStore()
```

```
3  const { profile, isLogin } = storeToRefs(userStore)
4  const { logout } = userStore
```

核心理解

Pinia store 本质上也是响应式对象，所以直接解构会面临和 `reactive` 一样的问题。

`storeToRefs()` 的价值，不是单纯让语法好看，而是把你解构出来的字段继续接回响应式系统。官方文档也明确说明，它类似 `toRefs()`，但会跳过方法和非响应式属性。

常见坑

1. 直接 `const { user } = useUserStore()`，然后以为它还会自动更新。
2. 把 actions 也一股脑放进 `storeToRefs()`。
3. 看到 store 解构失效，却误以为是 Pinia 有 bug。

面试展开

如果面试官问为什么 Pinia 要用 `storeToRefs()`，我会说：因为 store 本质上也是响应式对象，直接解构会绕开代理读取，响应性就断了。`storeToRefs()` 的作用，就是把这些字段重新包装成 refs，同时忽略方法和非响应式属性。

6.7 Store 设计要按边界，不要按“方便”

定位

这一节讲应用级状态应该怎么收边。

必会结论

1. 不是所有共享数据都应该进 store。
2. store 更适合承载跨页面、跨组件、跨会话流程里真正共享的业务状态。
3. 页面局部状态、组件临时状态、纯派生状态，不应无脑放进 Pinia。

核心理解

store 的价值在于收敛共享状态，不在于“任何地方都能拿到”。如果一个状态只在单页或单组件内部使用，把它塞进 Pinia 反而会增加心智成本。真正适合进 store 的，通常是用户会话、权限、全局筛选条件、应用配置、跨页面缓存和明确的业务聚合状态。store 一旦太大，就会变成另一种“全局变量中心”。

常见坑

1. 为了方便调试或复用，把局部状态也硬塞进 store。

2. store 之间互相引用过重，形成耦合网。
3. store 既持有源状态，又持有一堆可由 getters 推导出的冗余状态。

面试展开

更成熟的说法是：store 应该服务于共享边界，而不是服务于“哪里都能访问”。如果一个状态没有跨边界共享价值，我更倾向留在页面或组件内部。

6.8 SSR 场景下要强调每请求独立实例

定位

这一节把 Router、Pinia 和 SSR 拉到同一条工程主线上。

必会结论

1. SSR 场景下，app、router、pinia 都应该按请求创建独立实例。
2. 共享单例会带来跨请求状态污染。
3. Pinia 官方文档也明确强调要在正确上下文里使用 store，并在客户端先完成 state hydration。

代码抓手

代码块

```
1  import { createApp } from 'vue'
2  import { createPinia } from 'pinia'
3  export function createMyApp() {
4    const app = createApp(App)
5    const pinia = createPinia()
6    const router = createRouter(...)
7    app.use(pinia)
8    app.use(router)
9    return { app, pinia, router }
10 }
```

核心理解

SSR 的核心风险不在“能不能跑起来”，而在“多个用户请求会不会串状态”。服务端进程通常是常驻的，如果你把 app、router、pinia 或用户态共享状态放在模块级单例里，一个请求写进去的数据就可能被下一个请求读到。这个问题在面试里非常拉分，因为它体现你有没有真正理解同构环境。

常见坑

1. 把 store 单例直接定义在模块顶层。

2. 客户端还没完成 hydration 就先调用 store。
3. 忽略状态序列化和 XSS 风险。

面试展开

更稳的表达是：SSR 最大的状态风险不是响应式失效，而是跨请求共享单例造成的数据串用户。标准做法就是每个请求都创建独立的 app、router、pinia 实例，并在客户端先恢复初始状态再消费 store。

6.9 应用组织的主线不是“会配”，而是“会分层”

定位

这一节给第六章做收口。

必会结论

1. Router 管页面边界和导航时机。
2. Pinia 管共享状态边界。
3. 组件层只负责局部表现和局部交互。

核心理解

一个 Vue 应用如果不好维护，通常不是因为技术栈不够，而是分层不清。页面级逻辑该不该进路由，跨页面状态该不该进 Pinia，局部状态该不该留在组件，这些判断比“API 会不会写”更影响长期可维护性。真正成熟的 Vue 工程，会把页面层、状态层、组件层的职责拆得很清楚。

常见坑

1. Router、Pinia、组件三层职责混在一起。
2. 组件承受了太多页面级和应用级逻辑。
3. 路由和 store 只配了功能，却没有承担组织职责。

面试展开

如果让我总结第六章，我会说：Vue 应用组织的关键不在于会不会配 Router 和 Pinia，而在于能不能把页面边界、共享状态边界和组件表现边界拆清楚。

7 性能与现代 Vue

7.1 性能优化先分“加载性能”和“更新性能”

定位

这一节先把性能问题拆开，不然一上来就只会背优化技巧。

必会结论

1. 页面加载性能和更新性能不是同一类问题。
2. 加载性能关注首屏内容可见和可交互速度。
3. 更新性能关注状态变化后界面响应是否足够快。

核心理解

Vue 官方性能指南也明确把性能分成页面加载性能和更新性能两大类。前者关注架构选型、包体积、代码分割、SSR/SSG；后者关注 props 稳定性、更新边界、列表渲染、响应式开销和组件抽象层级。很多团队一提性能就开始微调渲染细节，但如果首屏是个臃肿的 SPA 包，问题根本不在更新层。

常见坑

1. 不分问题类型，加载慢和更新慢混在一起调。
2. 一上来就写很多细碎优化，却没先确认架构是否合适。
3. 忽视包体积、拆包和渲染模式，只盯组件 rerender。

面试展开

更成熟的性能回答，通常会先说：我会先判断问题发生在加载阶段还是更新阶段。加载性能优先看渲染模式、拆包和首屏 JS，更新性能优先看状态边界、列表、组件抽象和不必要的响应式开销。

7.2 加载性能先看架构和拆包

定位

这一节讲对页面加载最有影响的几个动作。

必会结论

1. 对首屏敏感的页面，不应无脑走纯 CSR SPA。
2. 路由级懒加载是 Vue 应用最基础的代码分割策略。
3. 重型组件、编辑器、图表、后台大页签，都适合异步加载。

核心理解

Vue 官方性能指南也强调，页面加载优化首先要选对架构。如果业务对 SEO、首屏内容可见和弱网表现敏感，SSR 或 SSG 往往比纯 CSR 更合适。其次是拆包，路由组件和重型组件都应该按需加载。不要把所有能力都塞进首屏入口里，然后再抱怨首屏慢。

常见坑

1. 所有页面打进同一个大包。
2. 编辑器、图表、富文本、AI 面板首屏全量加载。
3. 明明适合 SSR/SSG，却仍然坚持纯 SPA。

面试展开

如果被问加载性能，我会先从架构开始讲：首屏敏感场景先判断 CSR、SSR、SSG 是否选对，再看路由和重型组件有没有做懒加载，最后再看包体积和第三方库引入方式。

7.3 更新性能先看边界，而不是先看框架

定位

这一节把更新性能的起点放回“边界设计”。

必会结论

1. Vue 组件更新本身不是问题，更新范围失控才是问题。
2. props 是否稳定、状态是否最小化、组件是否过度抽象，直接决定更新成本。
3. 很多更新性能问题，本质上是边界设计不合理。

核心理解

Vue 官方指南专门强调 props 稳定性。一个子组件只要接收到的 props 变化了，就可能更新，所以真正重要的是“有没有把变化限制在最小边界内”。比如列表项如果每次都收到一个新的大对象或不稳定函数，就很容易整片重跑。性能优化的核心不是“别更新”，而是“只让真正需要变化的部分更新”。

代码抓手

代码块

```
1 <ListItem
2   v-for="item in list"
3   :id="item.id"
4   :active="item.id === activeId"
5 />
```

常见坑

1. 父组件每次都传新对象、新数组、新函数，导致子组件全更新。
2. 列表项收到的是过于宽泛的 props，更新粒度太粗。
3. 为了抽象而抽象，列表里套很多中间组件层。

面试展开

我一般会先说：更新性能问题最先要看边界。官方也强调 props 稳定性，因为一旦子组件输入不稳定，再好的 diff 也只是被动善后。

7.4 `v-if`、`v-show`、`v-once`、`v-memo` 都是策略，不是口诀

定位

这一节把几个常见性能指令讲回语义。

必会结论

1. `v-if` 适合低频切换或条件很少成立的内容。
2. `v-show` 适合高频显示隐藏切换。
3. `v-once` 适合纯静态内容。
4. `v-memo` 适合只在少量依赖变化时跳过子树更新。

核心理解

这些指令不是面试口诀，而是更新策略选择。`v-if` 是创建和销毁，`v-show` 是样式切换，成本模型完全不同。`v-once` 适合“真静态”，`v-memo` 适合你明确知道这个子树只应该在某些依赖变化时才重新评估。它们都很好用，但前提是你真的知道问题在哪。

常见坑

1. 把 `v-show` 用在几乎不出现的重型节点上。
2. 把 `v-if` 用在高频切换区域，导致反复卸载重建。
3. 看见性能问题就先加 `v-memo`，却没先确认更新边界。

面试展开

我会这样讲：这些指令不是通用加速器，而是条件渲染策略。先判断问题是频繁切换、静态内容、还是局部子树重复更新，再决定是否用它们。

7.5 大列表的核心不是 diff，而是 DOM 数量

定位

这一节把大列表性能问题讲到根上。

必会结论

1. 大列表最常见的瓶颈是 DOM 节点太多，而不是框架 diff 不够快。

2. 海量列表应优先考虑虚拟列表。
3. `key` 必须稳定且唯一，尽量使用业务 ID，而不是索引。

核心理解

无论框架多快，只要你一次性渲染几千上万个节点，浏览器都会吃力。Vue 官方性能指南也明确推荐对大型列表使用虚拟化。列表性能优化首先是减少真实 DOM 数量，其次才是让已有节点的更新更可控。与此同时，`key` 的稳定性直接影响复用效果和状态正确性，这是性能和正确性共同的问题。

常见坑

1. 用数组索引做 `key`。
2. 大量列表项里又套很多抽象组件。
3. 不做虚拟化，只指望 diff 算法“自己够快”。

面试展开

更成熟的表达是：大列表性能问题首先是 DOM 规模问题，其次才是 diff 成本问题。真正有效的方案往往是虚拟滚动、稳定 `key`、减少组件层级和减少不必要的响应式开销。

7.6 大对象和第三方实例要控制响应式开销

定位

这一节把响应式系统和性能问题真正接起来。

必会结论

1. Vue 默认深响应很方便，但大型不可变结构会带来额外追踪开销。
2. 第三方实例通常不适合被深度代理。
3. `shallowRef`、`shallowReactive`、`markRaw` 是这类场景的常用方案。

代码抓手

代码块

```
1 import { shallowRef } from 'vue'
2 const graphData = shallowRef<any[]>([])
3 function replaceGraph(next: any[]) {
4   graphData.value = next
5 }
```

核心理解

Vue 官方性能指南专门提到，应减少大型不可变数据的响应性开销。很多业务数据其实更接近“整体替换”的不可变结构，而不是“深层频繁局部修改”的对象树。对这类数据做深追踪，收益并不大。再比如图表、编辑器、地图实例，它们通常也不希望被代理干预。这时就应该明确改用浅响应或跳过代理。

常见坑

1. 大型 schema、图结构、字典树全量深代理。
2. 第三方实例塞进普通 `reactive` 对象。
3. 明明只关心整体替换，却还在深层追踪每个属性。

面试展开

我会把这一点讲成：性能优化不是只看 DOM，也要看状态层。对大对象和第三方实例，响应式边界设计得不好，更新成本和理解成本都会一起变高。

7.7 `KeepAlive`、`Teleport`、异步组件、`Suspense` 是现代 Vue 的高频能力

定位

这一节讲现代 Vue 应用里常见但容易讲浅的几个内置能力。

必会结论

1. `KeepAlive` 解决组件缓存，不是性能万能药。
2. `Teleport` 解决 DOM 挂载位置，不改变组件逻辑归属。
3. `defineAsyncComponent` 解决组件按需加载。
4. `<Suspense>` 负责协调异步依赖，但官方仍标记为实验性特性。

核心理解

这几个能力解决的是不同层面的问题。`KeepAlive` 用内存换切换速度，适合页签、路由缓存、复杂表单回退保留；`Teleport` 让弹窗、浮层这类 UI 脱离父层级的 DOM 约束，但 props、emit、provide/inject 逻辑关系不变；异步组件让重型能力按需加载；`Suspense` 则是在组件树层面协调多个异步依赖的等待态。它们都很重要，但不能互相混用概念。

代码抓手

代码块

```
1 import { defineAsyncComponent } from 'vue'
2 const AsyncPanel = defineAsyncComponent({
3   loader: () => import('./Panel.vue')
```

常见坑

1. 用 `KeepAlive` 却不管缓存上限和失活逻辑。
2. 以为 `Teleport` 会打断组件树通信。
3. 把 `<Suspense>` 当成绝对稳定的生产基础设施，却忽略它仍是实验性特性。

面试展开

如果面试官追问现代 Vue 内置能力，我会分层回答：`KeepAlive` 管缓存，`Teleport` 管挂载点，异步组件管拆包与按需加载，`Suspense` 管异步等待态协调。这四者解决的是不同问题，不应混成“性能优化套件”。

7.8 SSR、Hydration 和延迟激活是更高阶的性能话题

定位

这一节把现代 Vue 的性能视角从 CSR 推到更完整的交付链路。

必会结论

1. SSR 提升的是首屏内容可见和 SEO，不是简单“更快”一句话。
2. Hydration 是接管已有 DOM，而不是重新全量创建。
3. Vue 3.5+ 的异步组件支持 hydration strategy，可按 idle、visible、interaction 等时机延迟激活。

代码抓手

代码块

```
1  import { defineAsyncComponent, hydrateOnVisible } from 'vue'
2  const AsyncChart = defineAsyncComponent({
3    loader: () => import('./Chart.vue'),
4    hydrate: hydrateOnVisible({ rootMargin: '100px' })
5  })
```

核心理解

Vue 官方异步组件文档已经明确给出 3.5+ 的 hydration strategy。它的意义不是让 SSR 更“炫”，而是让某些非关键交互区域先输出 HTML，再按合适时机激活，平衡首屏、可交互时间和主线程压力。与此同时，也要知道 SSR 本身会带来更复杂的部署和一致性要求，Hydration mismatch、跨请求状态污染、浏览器专属 API 访问时机，这些都是真实成本。

常见坑

1. 把 SSR 理解成“服务端把页面重新做一遍，所以一定更快”。
2. 忽视 hydration mismatch 的一致性要求。
3. 看见延迟激活就全局滥用，反而影响关键交互体验。

面试展开

更成熟的说法是：SSR 优化的是首屏内容交付，而 hydration 负责把已有 HTML 接成可交互应用。Vue 3.5+ 的异步组件 hydration strategy 则进一步把“什么时候激活”也纳入可控范围，这是比普通拆包更进阶的性能手段。

7.9 性能优化的正确顺序

定位

这一节给整章做收口，避免把性能学成技巧清单。

必会结论

1. 先定位瓶颈，再选优化策略。
2. 先看架构，再看边界，再看局部技巧。
3. 不要把 Vue 性能优化学成“见问题就加缓存和 memo”。

核心理解

Vue 官方文档也强调，性能优化首先是知道如何测量。真正的优化顺序通常是：先确认问题发生在加载阶段还是更新阶段；再看是不是架构选型、包体积、拆包、路由边界或数据边界的问题；最后才轮到指令级和组件级细化优化。这个顺序很重要，因为大多数性能问题都不是靠一两个 API 名字解决的。

常见坑

1. 没测量就优化。
2. 先写一堆“优化代码”，后面连收益都说不清。
3. 把组件更新本身当 bug，而不是先判断更新是否合理。

面试展开

如果让我总结 Vue 性能优化，我会说：先分加载性能和更新性能，再从架构、边界和局部策略三层去查。Vue 的问题很少是“框架太慢”，更多是架构选型、状态边界或渲染边界没有设计好。

8 底层原理心智

8.1 Vue 3 宏观架构

定位

这一节先把 Vue 3 拆成几个真正工作的包，不然后面讲 `compiler-sfc`、`runtime-core`、`patch`、`diff` 时会全混在一起。

必会结论

1. Vue 3 不是一个大黑盒，而是一组职责清晰的包协作起来。
2. `@vue/reactivity` 负责响应式，`@vue/runtime-core` 负责平台无关运行时，`@vue/runtime-dom` 负责浏览器 DOM 落地。
3. `@vue/compiler-core`、`@vue/compiler-dom`、`@vue/compiler-sfc` 负责不同层级的编译。
4. `@vue/server-renderer` 负责 SSR 输出和激活相关能力。

核心理解

可以把 Vue 3 粗略看成三层。第一层是编译层，负责把人写的模板和 SFC 变成运行时能执行的代码；第二层是运行时核心，负责组件实例、VNode、调度器、`patch`；第三层是平台适配层，负责把运行时的抽象节点操作落到浏览器 DOM。这样理解后，你就不会再把 `compiler-sfc` 和 `runtime-core` 混成一类能力。

源码抓手

1. `packages/reactivity`
2. `packages/runtime-core`
3. `packages/runtime-dom`
4. `packages/compiler-core`
5. `packages/compiler-dom`
6. `packages/compiler-sfc`
7. `packages/server-renderer`

面试展开

更稳的说法是：Vue 3 是编译器和运行时强耦合协作的框架。编译器负责把模板里的静态和动态信息提前编码，运行时负责响应式驱动和节点更新，平台适配层负责真正操作 DOM，所以 Vue 3 的强项不是单点，而是整条链路都可控。

8.2 一份 `.vue` 文件是怎么变成页面的

定位

这一节回答最核心的问题之一：SFC 到底怎么落到页面。

必会结论

1. `.vue` 文件不是浏览器直接执行的格式。
2. `compiler-sfc` 会先把 SFC 拆成 descriptor，再分别处理 `template`、`script`、`style`。
3. `template` 会被编译成 render，运行时最终执行的是 render，不是模板字符串。

代码抓手

代码块

```
1 <template>
2   <button @click="count++">{{ count }}</button>
3 </template>
4 <script setup lang="ts">
5   import { ref } from 'vue'
6   const count = ref(0)
7 </script>
```

代码块

```
1 // 你真正要建立的心智，不是精确产物，而是这条链路
2 parseSFC(source)
3 -> compileScript(descriptor)
4 -> compileTemplate(descriptor.template)
5 -> 生成组件对象 + render
6 -> runtime 挂载组件
7 -> render 返回 vnode
8 -> patch 到 DOM
```

核心理解

SFC 处理大致分三步。第一步，`parse` 把 `.vue` 文件切成 `template`、`script`、`style` 等块；第二步，`compileScript` 处理普通 `script` 和 `script setup`，把编译期宏转成标准组件逻辑；第三步，`compileTemplate` 把模板转成 render。到了运行时，组件实例执行 `setup`，render 返回 VNode，renderer 再把 VNode 挂载到 DOM。也就是说，SFC 是开发时格式，render 才是运行时入口。

源码抓手

1. `packages/compiler-sfc/src/parse.ts`

2. `packages/compiler-sfc/src/compileScript.ts`
3. `packages/compiler-sfc/src/compileTemplate.ts`

常见坑

1. 把 `<script setup>` 误解成新的运行时 API。
2. 把 `compiler-sfc` 误叫成 `runtime-sfc`。
3. 以为模板更新是运行时字符串替换。

面试展开

如果面试官问 SFC 到页面的链路，我会说：SFC 先由 `compiler-sfc` 解析，`script setup` 在编译期展开，模板编译成 `render`，运行时创建组件实例并执行 `render` 生成 `VNode`，再由 `renderer` 挂载和更新 DOM。Vue 真正执行的不是 `.vue` 文件本身，而是编译后的组件定义和 `render` 函数。

8.3 模板编译主线：`parse`、`transform`、`generate`

定位

这一节不是讲 AST 字段，而是讲编译器为什么对 Vue 3 这么重要。

必会结论

1. 模板编译不是简单语法翻译，而是静态分析加代码生成。
2. 编译主线可以压成三步：`parse -> transform -> generate`。
3. Vue 3 的很多性能优化都来自 `transform` 阶段提前提纯信息。

代码抓手

代码块

```
1  // 心智模型，不是源码原样
2  template
3  -> parse(template)      // 生成 AST
4  -> transform(ast)       // 转换指令、分析静态节点、打优化标记
5  -> generate(ast)        // 生成 render 函数字符串
```

核心理解

`parse` 负责把模板变成 AST；`transform` 负责处理指令语义、节点转换和静态分析；`generate` 负责输出 `render` 代码。真正拉开差距的是 `transform`，因为 Vue 3 会在这个阶段做静态提升、`PatchFlag` 标记、`Block Tree` 相关信息收集，这让运行时拿到的不是“纯 `VNode` 生成代码”，而是“已经带着优化提示的 `VNode` 生成代码”。

源码抓手

1. `packages/compiler-core/src/parse.ts`
2. `packages/compiler-core/src/transform.ts`
3. `packages/compiler-core/src/codegen.ts`
4. `packages/compiler-dom/src/transforms/*`

常见坑

1. 以为编译器只是把模板换成 `h()` 调用。
2. 讲 Vue 3 性能时只讲 `Proxy`，不讲编译期优化。
3. 见到 AST 就陷入字段细节，反而讲不清主线。

面试展开

更好的表达是：Vue 模板编译不是单纯翻译语法，而是会在编译阶段提前分析哪些节点静态、哪些属性动态、哪些子树值得关注，然后把这些信息直接写进生成结果里，让运行时少做很多无意义工作。

8.4 响应式系统到底怎么工作

定位

这一节正式进入 Vue 3 的底层主叙事：`ReactiveEffect + track + trigger`。

必会结论

1. Vue 3 响应式的底层主线应该讲 `ReactiveEffect`，不该再以 `Dep/Watcher` 作为主叙事。
2. 核心机制是读取时 `track`，写入时 `trigger`。
3. 组件 `render`、`computed`、`watch`，本质上都建立在 `effect` 机制上。

代码抓手

代码块

```
1  let activeEffect: ReactiveEffect | undefined
2  const targetMap = new WeakMap<object, Map<PropertyKey, Set<ReactiveEffect>>>>()
3
4  function track(target: object, key: PropertyKey) {
5    if (!activeEffect) return
6    let depsMap = targetMap.get(target)
7    if (!depsMap) {
8      depsMap = new Map()
9      targetMap.set(target, depsMap)
10   }
```

```

11     let dep = depsMap.get(key)
12     if (!dep) {
13         dep = new Set()
14         depsMap.set(key, dep)
15     }
16     dep.add(activeEffect)
17 }
18
19 function trigger(target: object, key: PropertyKey) {
20     const depsMap = targetMap.get(target)
21     if (!depsMap) return
22     const dep = depsMap.get(key)
23     dep?.forEach(effect => {
24         if (effect.scheduler) effect.scheduler()
25         else effect.run()
26     })
27 }

```

核心理解

这套模型里，`WeakMap` 负责按 target 分桶，`Map` 负责按 key 精确映射，`Set` 负责存放 effect 并去重。组件 render 读取了哪些状态，这些状态就会把当前 render effect 收进去；状态变化时再把这些 effect 挑出来重跑。这个模型统一解释了“为什么读取才会建立依赖”“为什么解构会断响应性”“为什么组件更新、computed、watch 能共享底层机制”。

源码抓手

1. `packages/reactivity/src/effect.ts`
2. `packages/reactivity/src/reactive.ts`
3. `packages/reactivity/src/ref.ts`
4. `packages/reactivity/src/baseHandlers.ts`

常见坑

1. 还在把 Vue 3 主体讲成 Vue 2 的 `Dep` 和 `Watcher`。
2. 以为响应式会自动感知所有业务语义，忽略依赖是在读取时建立的。
3. 把 `reactive` 解构丢响应性只记成结论，不知道为什么。

面试展开

我通常会说：Vue 3 的响应式本质上是一个依赖图模型，核心是 `ReactiveEffect + track + trigger`。数据被读取时收集当前活跃 effect，数据被修改时触发对应 effect 或交给调度器，这样组件渲染、计算属性和侦听器都能统一在一套机制里理解。

8.5 computed、watch、watchEffect 在底层上的角色

定位

这一节不是复述 API 用法，而是解释它们为什么行为不同。

必会结论

1. `computed` 本质上是带缓存、带脏标记的惰性 effect。
2. `watch` 是显式监听源上的副作用封装。
3. `watchEffect` 是立即执行并自动收集同步依赖的 effect 封装。

代码抓手

代码块

```
1  // computed 的心智模型
2  let dirty = true
3  let cachedValue: unknown
4  const runner = new ReactiveEffect(getter, () => {
5    dirty = true
6  })
7  return {
8    get value() {
9      if (dirty) {
10        cachedValue = runner.run()
11        dirty = false
12      }
13      return cachedValue
14    }
15  }
```

核心理解

`computed` 的关键不是“能算值”，而是它把 getter 包成 lazy effect，并用 `dirty` 控制何时重算。`watch` 的关键不是“能监听”，而是它把依赖源、回调、副作用清理、调度时机收束成一个明确协议。`watchEffect` 则更接近“直接跑一个 effect，并在过程中自动记住依赖”。讲到底层时，这三者一定要回到 effect 模型上，否则就会只剩 API 差异表。

源码抓手

1. `packages/reactivity/src/computed.ts`
2. `packages/runtime-core/src/apiWatch.ts`

常见坑

1. 只会背 `computed/watch/watchEffect` 区别，却讲不出底层共性。
2. 误以为 `watchEffect` 会天然追踪 `await` 之后访问的依赖。
3. 不知道为什么 `computed` 能缓存而普通函数不能。

面试展开

更好的答法是：三者底层都和 effect 有关，但封装目标不同。`computed` 强调惰性和缓存，`watch` 强调显式来源和副作用控制，`watchEffect` 强调自动依赖收集和立即执行。

8.6 runtime-core 在做什么

定位

这一节专门补你最关心的点：`runtime-core` 到底负责什么。

必会结论

1. `runtime-core` 是平台无关的运行时核心，不直接关心浏览器 DOM 细节。
2. 它负责组件实例、props/emits/slots、生命周期、render effect、scheduler、VNode、patch 主流程。
3. `runtime-dom` 只是在它之上提供浏览器环境的宿主操作实现。

代码抓手

代码块

```
1  // 你要建立的是这个分层心智
2  createRenderer({
3    createElement,
4    insert,
5    remove,
6    patchProp,
7    setElementText
8  })
```

核心理解

`runtime-core` 可以理解成“如果不考虑具体平台，Vue 组件运行起来需要什么”。答案包括：怎么创建组件实例，怎么初始化 props 和 slots，怎么执行 `setup`，怎么创建 render effect，怎么比较新旧 VNode，怎么调度更新。`runtime-dom` 再把这些抽象动作翻译成浏览器里的

`document.createElement`、`insertBefore`、属性 `patch` 等宿主操作。所以 `runtime-core` 讲的是框架逻辑，`runtime-dom` 讲的是浏览器适配。

源码抓手

1. `packages/runtime-core/src/component.ts`
2. `packages/runtime-core/src/componentProps.ts`
3. `packages/runtime-core/src/componentSlots.ts`
4. `packages/runtime-core/src/componentRenderUtils.ts`
5. `packages/runtime-core/src/vnode.ts`
6. `packages/runtime-core/src/renderer.ts`
7. `packages/runtime-core/src/scheduler.ts`

常见坑

1. 以为 `runtime-core` 就是 DOM 更新代码。
2. 分不清 `runtime-core` 和 `runtime-dom` 的边界。
3. 讲运行时只会讲 `VNode`，不会讲组件实例和调度器。

面试展开

如果面试官问 `runtime-core`，比较稳的回答是：它是 Vue 运行时的抽象内核，负责组件实例、`VNode`、`render effect`、生命周期和 `patch` 主流程，但并不直接操作 DOM；DOM 相关宿主能力由 `runtime-dom` 注入。

8.7 `VNode`、`renderer` 与 `patch`

定位

这一节讲“为什么 Vue 最终一定会落到 `VNode` 和 `patch`”。

必会结论

1. `VNode` 是运行时描述 UI 的中间结构。
2. `renderer` 的职责是把 `VNode` 树挂载、更新、卸载到具体平台。
3. `patch` 是新旧 `VNode` 之间的差异应用过程，不等于“全树暴力比对”。

代码抓手

代码块

```
1  const vnode = {
```

```
2   type: 'div',
3   props: { class: 'box' },
4   children: 'hello'
5 }
```

核心理解

模板和 render function 最终都要产出 VNode。renderer 拿到 VNode 后，首次挂载时走 mount 流程，更新时走 patch 流程。patch 的本质是：看新旧节点是否同类型、是否可复用、子节点是什么形态，再决定是复用、替换、更新属性还是递归处理 children。这里一定要注意，Vue 3 的 patch 不是裸奔的，它吃到了编译器提前给出的优化信息，所以更新成本远低于“完全不知道哪些地方动态”的纯运行时 VDOM。

源码抓手

1. `packages/runtime-core/src/vnode.ts`
2. `packages/runtime-core/src/renderer.ts`
3. `packages/runtime-dom/src/nodeOps.ts`
4. `packages/runtime-dom/src/patchProp.ts`

常见坑

1. 把 VNode 当成 Vue 的最终目标，而不是中间表示。
2. 以为 patch 就是每次全树递归比较所有节点。
3. 只会讲虚拟 DOM，不会讲 renderer 和宿主适配。

面试展开

我会这样讲：VNode 是对目标 UI 的运行时描述，renderer 负责把这种描述落到具体平台，patch 负责把新旧描述之间的差异高效同步到真实 DOM。Vue 3 的关键不是只有虚拟 DOM，而是“带编译信息的 VNode 更新”。

8.8 diff 算法与 `key`

定位

这一节不走教学版“简单 diff 完整推导”，而是讲 Vue 3 真正重要的 keyed children 更新主线。

必会结论

1. `key` 的本质是节点稳定身份，不只是为了消除警告。
2. Vue 3 列表 diff 的重点是 keyed children 的高效更新。

3. 真正的主线是头尾同步、中段映射、复用/卸载、LIS 减少移动。

代码抓手

代码块

```
1 // 心智模型，不是源码原样
2 1. 从头同步相同前缀
3 2. 从尾同步相同后缀
4 3. 对中间未知区间建立 key -> newIndex 映射
5 4. 遍历旧节点，决定复用还是卸载
6 5. 生成 newIndexToOldIndexMap
7 6. 对这个序列求 LIS
8 7. LIS 保持不动，其余节点再移动或挂载
```

核心理解

很多教程喜欢从简单 diff、双端 diff 开始讲，这可以做背景，但 Vue 3 真正值得掌握的是 `patchKeyedChildren` 这条主线。为什么要先做头尾同步？因为很多列表变更实际只发生在中间。为什么要建 `key -> newIndex` 映射？因为这样复用旧节点时能快速定位新位置。为什么要求 LIS？因为它能找出“已经相对有序、无需移动”的最大节点子集，从而只移动那些真正必须移动的 DOM。

代码抓手

代码块

```
1 // 例子：新顺序对应旧索引
2 [2, 3, 5, 4]
3 // LIS 是 [2, 3, 5]
4 // 说明前三个相对有序，只需要移动 4 对应的节点
```

源码抓手

1. `packages/runtime-core/src/renderer.ts`
2. 重点函数：`patchKeyedChildren`

常见坑

1. 把 `key` 当成“Vue 要求必须写的语法糖”。
2. 列表里用数组索引当 `key`。
3. 以为 Vue 3 的 diff 还是“逐个暴力找和搬”。

面试展开

我通常会说：Vue 3 在 keyed children 更新时，不是简单暴力搬运所有节点，而是先做前后缀同步，再对中段建立映射，最后通过最长递增子序列尽量保留稳定节点，只移动真正必须移动的节点。这才是 `key` 和 LIS 在 Vue 3 里的实际价值。

8.9 编译优化为什么能让 Vue 3 更快

定位

这一节专门回答高频问题：“Vue 3 为什么普遍比 Vue 2 快？”

必会结论

1. 不能把 Vue 3 变快只归因于 `Proxy`。
2. Vue 3 的性能提升来自响应式重构、编译优化、运行时精准更新三层叠加。
3. 编译器提供的 PatchFlag、静态提升、Block Tree 是关键差异。

代码抓手

代码块

```
1  createElementVNode(  
2    "div",  
3    { class: _normalizeClass(_ctx.cls) },  
4    _toDisplayString(_ctx.msg),  
5    3 /* TEXT | CLASS */  
6  )
```

核心理解

Vue 官方渲染机制文档里把这件事讲得很清楚：Vue 3 不是纯运行时 VDOM，而是 compiler-informed virtual DOM。具体来说，静态提升会把纯静态节点提到 render 外复用；PatchFlag 会告诉运行时“这里只需要关心文本、class 或 props”；Block Tree 会把动态后代节点平铺到 `dynamicChildren` 里，更新时不必深度遍历整棵树。于是 Vue 3 的更新不是“每次从头检查一遍”，而是“带着地图走捷径”。

源码抓手

1. `packages/compiler-core/src/transforms/*`
2. `packages/runtime-core/src/vnode.ts`
3. `packages/runtime-core/src/renderer.ts`

常见坑

1. 把 Vue 3 快只答成“因为 Proxy 更先进”。
2. 不知道 PatchFlag 和 Block Tree 的存在。
3. 把虚拟 DOM 一概理解成全树 brute force diff。

面试展开

我会这样答：Vue 3 更快不是单点优化。第一层是响应式更现代；第二层是编译器会做静态提升、PatchFlag、Block Tree 等优化，把很多运行时工作前置；第三层是运行时 diff 结合这些信息做更精准更新，所以整体性能才上去了。

8.10 调度器和 nextTick

定位

这一节讲状态变化到 DOM 更新之间为什么不是同步直达。

必会结论

1. Vue 不会每次状态变化都立刻同步 patch DOM。
2. 更新任务会进入 scheduler 队列，做去重和批量刷新。
3. nextTick 的意义是等待本轮已入队更新刷完后的 DOM 状态。

代码抓手

代码块

```
1  const queue: Function[] = []
2  function queueJob(job: Function) {
3    if (!queue.includes(job)) queue.push(job)
4    queueFlush()
5  }
6  function flushJobs() {
7    for (const job of queue) job()
8  }
```

核心理解

如果每次状态变化都直接同步更新 DOM，连续多次修改就会造成大量重复工作。Vue 的调度器会把更新 effect 入队、去重，然后在合适时机统一刷新。nextTick 不是“延迟一下”的工具，而是“等本轮 Vue 更新队列完成”的承诺。这一点把侦听器 flush 时机、组件更新节奏、DOM 访问时机全串起来了。

源码抓手

1. `packages/runtime-core/src/scheduler.ts`
2. `packages/runtime-core/src/renderer.ts`

常见坑

1. 把 `nextTick` 当通用异步工具滥用。
2. 不理解为什么改了状态后立刻读 DOM 还是旧的。
3. 以为 Vue 的响应式更新是逐条同步打到界面上的。

面试展开

更稳的表达是：Vue 更新是响应式触发加调度器批处理，不是数据一变 DOM 就同步一变。

`nextTick` 本质上是等待当前这轮 job queue flush 之后再拿 DOM。

8.11 SSR、CSR 与 Hydration 高阶补充

定位

这一节只讲第八章需要的底层心智，不展开完整 SSR 工程实践。

必会结论

1. CSR 是客户端下载 JS 后生成 DOM，SSR 是服务端先产出 HTML，再由客户端激活。
2. Hydration 不是重新挂载整棵树，而是把事件和实例接到已有 DOM 上。
3. Hydration mismatch 和跨请求状态污染，是 SSR 面试里的高频拉分点。

核心理解

Vue 官方 SSR 文档明确强调，hydration 的关键是“接管已有 DOM”。如果服务端输出的 HTML 和客户端首屏 VNode 不一致，就会出现 mismatch，常见来源包括随机数、时间戳、非法 HTML、浏览器专属 API、初始状态不一致。另一个更偏工程的问题是跨请求状态污染，服务端若把 app、router、store 放成单例，不同用户请求就可能串数据。这些问题比“SSR 更快”更能体现你有没有真正理解 Vue 的服务端渲染。

源码抓手

1. `packages/server-renderer`
2. `packages/runtime-core/src/hydration.ts`

常见坑

1. 把 hydration 理解成“客户端再渲染一遍”。

2. 只讲 SSR 的优点，不讲一致性和状态隔离成本。
3. 忽略浏览器 API 只能在客户端生命周期里访问。

面试展开

我一般会这样讲：SSR 解决的是首屏 HTML 交付，hydration 解决的是把这段 HTML 接成可交互应用。真正难的不是 `renderToString` 本身，而是保证服务端和客户端首屏结构一致，以及每个请求状态隔离。

8.12 第八章速记

定位

这一节给你面试前快速回忆主线。

必会结论

1. Vue 3 要按“编译器 + 响应式 + 运行时”三段来记。
2. `compiler-sfc` 处理 `.vue`，`runtime-core` 管组件运行，`runtime-dom` 管 DOM 落地。
3. 响应式主线是 `ReactiveEffect + track + trigger`。
4. 更新主线是 render 生成新 VNode，再由 patch 做精准更新。
5. 性能主线是编译期优化信息加运行时精准 diff。

核心理解

一句话压缩：Vue 3 的核心不是一堆 API，而是一条从 SFC 编译、到响应式驱动、到 `runtime-core` 调和 patch、再到 DOM 或 hydration 落地的完整链路。只要这条线记住，很多细节都会自己归位。

面试展开

如果让我用 1 分钟讲完第八章，我会说：Vue 3 先通过 `compiler-sfc` 和模板编译把 SFC 变成 render；运行时由 `runtime-core` 创建组件实例、执行 `setup`、建立 render effect；响应式层通过 `track/trigger` 驱动重新渲染；renderer 再结合 PatchFlag、Block Tree 和 keyed diff 做精准更新；SSR 场景下则由 `server-renderer` 输出 HTML，再在客户端做 hydration 接管。

8.13 源码阅读路线

定位

这一节不是再讲原理，而是告诉你源码应该怎么读，避免一上来就在 `renderer.ts` 里迷路。

必会结论

1. 先按问题读，不要按文件树漫游。

2. 先建立链路，再进函数细节。
3. `runtime-core/src/renderer.ts` 是重头戏，但不应该成为第一站。

核心理解

更稳的源码阅读顺序通常是：

1. 先读 `packages/compiler-sfc`，知道 SFC 是怎么拆的。
2. 再读 `packages/reactivity`，知道 effect、track、trigger 怎么跑。
3. 再读 `packages/runtime-core/src/component.ts` 和 `vnode.ts`，知道组件实例和 VNode 长什么样。
4. 再进 `packages/runtime-core/src/renderer.ts`，看 mount、patch、component update、`patchKeyedChildren`。
5. 最后读 `scheduler.ts`、`hydration.ts`、`server-renderer`，补调度和 SSR。

面试展开

如果被问“你怎么看 Vue 源码”，我会说：我不会一上来通读所有文件，而是先按链路拆问题。先搞清编译怎么产出 render，再搞清响应式怎么驱动 effect，再搞清 runtime-core 怎么创建组件实例和执行 patch，最后再深入 diff、scheduler 和 hydration。这样读源码更容易形成完整心智，而不是只记住一堆文件名。

9 Vue 面试题与分层展开思路

9.1 这一章怎么用

定位

这一章不是八股速答表，而是按面试真实节奏来写：先给 `3-5 年` 能答稳的版本，再补 `5-7 年` 能拉开差距的展开，最后把高频追问也一起答掉。

必会结论

1. 这一章代码会比前面稍多一点，也会稍微细一点，但仍然只保留最能帮助回答追问的最小必要代码。
2. 每道题都要先能答对，再能答深，最后还要能接住追问。
3. 如果一题只能背定义，不能顺着讲到底层和工程边界，那面试里很容易被追问打断。

9.2 题目：Vue 生命周期怎么理解？

考点

组件实例初始化、render 执行时机、DOM 可用时机、更新后时机、卸载清理、KeepAlive 生命周期。

3-5 年回答

Vue 生命周期就是组件从创建、挂载、更新到卸载的一组阶段钩子。Vue 3 里最常用的是 `setup`、`onMounted`、`onUpdated`、`onUnmounted`。`setup` 适合初始化逻辑，`onMounted` 适合访问 DOM 和模板 ref，`onUpdated` 表示组件更新完成后的阶段，`onUnmounted` 适合清理定时器、事件和订阅。如果组件被 `KeepAlive` 缓存，还会有 `onActivated` 和 `onDeactivated`。

5-7 年展开

生命周期不要背中文表格，而要按“组件实例、render、DOM”三个时机理解。Vue 会先创建组件实例、解析 props、初始化 slots、执行 `setup`，这时组件逻辑入口已经开始了，但 DOM 还没挂上去。随后 render 执行，生成子树 vnode，renderer 完成挂载后才会进入 `onMounted`。状态变化后，组件的 render effect 重新执行，patch 完成后才进入 `onUpdated`。KeepAlive 下组件不是每次都真的销毁，而是进入缓存树，所以会有激活和失活钩子。

代码抓手

代码块

```
1  <script setup lang="ts">
2  import {
3    onMounted,
4    onUpdated,
5    onUnmounted,
6    onActivated,
7    onDeactivated,
8    useTemplateRef
9  } from 'vue'
10 const inputRef = useTemplateRef<HTMLInputElement>('inputRef')
11 onMounted(() => {
12   inputRef.value?.focus()
13 })
14 onUpdated(() => {
15   console.log('本轮 DOM 已更新')
16 })
17 onUnmounted(() => {
18   console.log('清理副作用')
19 })
20 onActivated(() => {
21   console.log('从缓存中恢复')
22 })
23 onDeactivated(() => {
24   console.log('进入缓存或被卸载')
```

```
25   })
26   </script>
27   <template>
28     <input ref="inputRef" />
29   </template>
```

高频追问与回答

1. `setup` 能不能拿到 `this`，为什么？

答：不能。因为 `setup` 执行时，组件公开实例代理还没建立，Vue 3 也不希望你继续用 `this` 组织逻辑。组合式 API 的设计目标就是让逻辑直接基于变量、函数和闭包来组织。

2. 为什么 `onMounted` 能访问 DOM，但 `setup` 里不应该默认访问 DOM？

答：因为 `onMounted` 触发时，组件已经完成首次挂载，模板对应的真实 DOM 已经存在；而 `setup` 发生在 render 和挂载之前，这时 DOM 通常还不可用。

3. `onMounted` 和 `nextTick` 怎么区分？

答：`onMounted` 解决的是“首次挂载完成后做什么”，`nextTick` 解决的是“当前这轮更新刷完后做什么”。前者是生命周期阶段，后者是调度时机工具。

4. `KeepAlive` 下会不会触发 `onUnmounted`？

答：正常切换缓存组件时，不会立刻触发 `onUnmounted`，而是触发 `onDeactivated`。只有组件真正从缓存中被移除、彻底销毁时，才会进入卸载生命周期。

易错点

1. 把 `setup` 理解成 DOM 已可用阶段。
2. 把 `onMounted` 和 `nextTick` 混成一个概念。
3. 以为 `KeepAlive` 缓存组件切换时一定会重新挂载。

9.3 题目：`ref` 和 `reactive` 应该怎么选？

考点

状态建模、值语义与对象语义、响应式边界、解构丢响应性。

3-5 年回答

`ref` 适合基本类型，也可以包对象；`reactive` 适合对象、数组这类引用类型。一般来说，单个值、布尔开关、计数器、可整体替换的数据，更适合用 `ref`；一组强相关字段组成的状态对象，更适合用 `reactive`。如果 `reactive` 要解构，通常要配合 `toRef` 或 `toRefs`，不然容易丢响应性。

5-7 年展开

真正的选择标准不是“哪个更高级”，而是状态形状和边界。`ref` 的好处是值语义更明确，重赋值更自然，跨函数传递和 TypeScript 推断通常也更稳；`reactive` 的好处是对象语义自然，适合一组字段作为整体容器来读写。但 `reactive` 的响应性来自代理对象本身，一旦直接解构，就会绕开代理读取，所以响应性会断掉。也正因为这个特性，很多团队在组件内会优先用 `ref` 建模，把 `reactive` 留给真正有整体对象语义的状态容器。

代码抓手

代码块

```
1  import { ref, reactive, toRefs } from 'vue'
2  const count = ref(0)
3  const state = reactive({
4    name: 'miaoma',
5    age: 7
6  })
7  const { name, age } = toRefs(state)
8  count.value++
9  name.value = 'vue'
```

高频追问与回答

1. 为什么 `reactive` 解构后会丢响应性？

答：因为依赖收集发生在“通过代理对象读取属性”的那一刻。你直接解构后拿到的是当前值，而不是后续仍然经过代理 getter 的访问，所以响应性就断掉了。

2. `ref` 包对象和 `reactive` 包对象有什么差别？

答：`ref` 包对象后，`.value` 里的对象会继续被转成响应式代理，所以两者底层都会接到响应式系统上。区别更多在于使用形态：一个是显式 `.value` 的值容器，一个是直接属性访问的代理对象。

3. 为什么很多团队更偏向先用 `ref`？

答：因为它更显式，重赋值更稳定，跨 composable 传递更自然，TS 推断也更直接。对象状态如果不是必须作为整体容器使用，很多时候拆成多个 `ref` 反而更清楚。

4. `toRef` 和 `toRefs` 什么时候用？

答：当你需要从响应式对象里安全地抽字段出来继续传递或解构时用。`toRef` 适合单个字段，`toRefs` 适合整体安全解构。

易错点

1. 把 `reactive` 当成“比 `ref` 更高级”的选择。

2. 所有状态都堆进一个大 `reactive` 对象。
3. 直接解构 store 或 `reactive` 后还以为它会自动更新。

9.4 题目： `computed`、`watch`、`watchEffect` 应该怎么区分？

考点

派生状态、副作用、自动依赖收集、缓存、侦听边界。

3-5 年回答

`computed` 用来做派生状态，适合根据已有响应式状态算出一个新值，而且它有缓存。`watch` 用来监听明确的数据源，数据变化后执行副作用，比如发请求、同步外部状态。`watchEffect` 会立即执行，并自动收集内部用到的响应式依赖，适合依赖源分散但副作用本身比较直接的场景。

5-7 年展开

这三者底层都和 effect 有关，但封装目标不一样。`computed` 本质上是带惰性求值和脏标记的 effect，它解决的是“一个值如何由其他值推导出来”，所以应该保持纯净。`watch` 是显式依赖源上的副作用封装，它强调新旧值、清理时机、调度控制。`watchEffect` 更接近“立即跑一个 effect，并在同步执行过程中自动记录依赖”，所以更适合轻量副作用，不适合特别复杂的异步控制流。

代码抓手

代码块

```
1  import { ref, computed, watch, watchEffect } from 'vue'
2  const price = ref(10)
3  const count = ref(2)
4  const total = computed(() => price.value * count.value)
5  watch(() => count.value, (newVal, oldVal) => {
6    console.log('count changed', oldVal, newVal)
7  })
8  watchEffect(() => {
9    document.title = `总价: ${total.value}`
10 })
```

高频追问与回答

1. 为什么 `computed` 有缓存？

答：因为它底层是一个 lazy effect，会用脏标记控制是否重新求值。依赖没变时直接返回上次结果，依赖变了才标脏并在下次读取时重算。

2. 为什么 `watchEffect` 不适合复杂异步？

答：因为它只会追踪同步执行阶段访问到的依赖。`await` 之后再读取的状态不会被自动收集，而且复杂异步逻辑也更容易看清依赖来源和清理边界。

3. 为什么很多场景推荐用 `computed` 替代“`watch` + 赋值”？

答：因为如果一个值本质上是派生状态，就应该用声明式方式表达推导关系，而不是用 `watch` 手工同步一份新状态。后者更容易造成双写和状态漂移。

4. `watch` 里怎么处理竞态请求？

答：要么使用 `onWatcherCleanup()`，要么用回调参数里的 `onCleanup`，在侦听器失效时取消上一次请求，避免旧请求回来的结果覆盖新状态。

易错点

1. 在 `computed` 里发请求、改 DOM、改状态。
2. 用 `watch` 维护本可以由 `computed` 推导的值。
3. 以为 `watchEffect` 会自动追踪 `await` 后面的依赖。

9.5 题目： `setup` 和 `<script setup>` 到底是什么关系？

考点

Composition API、编译期语法糖、组件逻辑入口、宏展开。

3-5 年回答

`setup()` 是 Vue 3 组件逻辑的核心入口，很多响应式状态、计算属性、侦听器和生命周期注册都在这里组织。`<script setup>` 只是 `setup()` 的编译期语法糖，它让写法更简洁，也更适合配合 `defineProps`、`defineEmits`、`defineModel` 这些宏。

5-7 年展开

这里最容易混淆的是“语法糖”和“运行时机制”两个层次。`setup` 是运行时概念，它决定了组件初始化时逻辑从哪里开始；`<script setup>` 是编译期概念，它把顶层代码编译到 `setup` 语义里，同时展开宏、减少样板代码、增强类型推断。所以 `script setup` 不是另一套组件系统，它只是把现代 Vue 最常见的 `setup` 写法前移到编译期处理了。

代码抓手

代码块

```
1  <script setup lang="ts">
2  import { ref, computed } from 'vue'
3  const props = defineProps<{ price: number }>()
4  const count = ref(1)
5  const total = computed(() => props.price * count.value)
```

高频追问与回答

1. 为什么说 `<script setup>` 是编译期语法糖？

答：因为它本身不会作为浏览器里新的运行时 API 存在，而是会在编译阶段被转换成标准组件定义和 `setup()` 逻辑。

2. `defineProps`、`defineEmits`、`defineModel` 为什么叫宏？

答：因为它们不是普通运行时函数，而是编译器识别并展开的特殊语法入口。

3. `setup` 和 Vue 2 的 `created` 是一回事吗？

答：不能简单等同。它们都属于很早的初始化阶段，但 Vue 3 的 `setup` 是组合式逻辑入口，且没有 `this`，也不意味着 DOM 可用。

4. 什么情况下不用 `<script setup>`？

答：极少数场景，比如你需要非常细地控制组件选项组织或使用非 SFC 形式时。但在现代工程里，默认还是推荐用 `<script setup lang="ts">`。

易错点

1. 把 `<script setup>` 当成新的运行时 API。

2. 把所有宏都当成普通函数来理解。

3. 进入 Vue 3 之后还完全停留在 Vue 2 的 Options API 组织方式。

9.6 题目：`v-if` 和 `v-show` 怎么选？为什么 `v-for` 一定要写 `key`？

考点

条件渲染策略、列表更新、节点身份、diff 复用。

3-5 年回答

`v-if` 是条件成立时才创建，不成立就销毁；`v-show` 是始终保留 DOM，只是切换 `display`。所以高频切换更适合 `v-show`，很少出现或初始不需要的内容更适合 `v-if`。`v-for` 要写 `key`，是因为 `key` 能告诉 Vue 每个节点的稳定身份，方便复用和正确更新。

5-7 年展开

`v-if` 和 `v-show` 的差别不只是“一个显示一个隐藏”，而是成本模型完全不同。`v-if` 对应的是挂载和卸载，`v-show` 对应的是样式切换。真正的选择标准要看切换频率和节点体量。`key` 更不是“为了消除警告”，它是列表 diff 的稳定身份标记。Vue 3 在 keyed children 更新时，会基于 `key`

建立映射、复用旧节点、卸载无用节点，并结合 LIS 减少 DOM 移动；如果你用索引当 `key`，节点身份就会跟着位置漂移，既影响性能，也容易影响组件内部状态正确性。

代码抓手

代码块

```
1 <li v-for="item in list" :key="item.id">
2   {{ item.title }}
3 </li>
```

高频追问与回答

1. 为什么不推荐用索引当 `key`？

答：因为列表一旦插入、删除、重排，索引会变化，节点身份就会错位，Vue 可能错误复用旧 DOM 或组件实例，导致状态和渲染结果不稳定。

2. `v-if` 和 `v-for` 为什么不建议写在同一节点上？

答：因为这样既增加模板语义复杂度，也让渲染边界变得不直观。更稳的做法通常是先用 `computed` 过滤数据，再交给 `v-for` 渲染。

3. `key` 只影响性能吗？

答：不是。它同时影响节点复用策略和组件实例身份，所以也会影响正确性。

4. `v-show` 一定更快吗？

答：不一定。如果节点本身很重，但切换很少，长期保留 DOM 和子组件反而可能更浪费资源。这还是要看切换频率和节点体量。

易错点

1. 认为 `v-show` 永远比 `v-if` 好。

2. 把 `key` 理解成单纯的性能字段。

3. 列表里直接用索引做 `key`。

9.7 题目：nextTick 是干什么的？

考点

scheduler、批量更新、DOM 刷新时机、侦听器 flush。

3-5 年回答

`nextTick` 用来等待 Vue 完成本轮 DOM 更新。因为 Vue 修改状态后不会每次都立刻同步更新 DOM，而是会把更新放进队列统一处理，所以如果你改完状态后要拿更新后的 DOM，就可以用

`nextTick`。

5-7 年展开

`nextTick` 的本质不是“延迟一下”，而是“等待当前这轮 scheduler 队列 flush 完成”。Vue 会把组件更新 effect 和部分相关 job 入队、去重、批量刷新，避免同一轮事件循环里重复 patch 同一个组件。也正因为这样，状态变了不代表 DOM 立刻就是新的。`nextTick` 解决的是渲染时机问题，不是业务异步问题。

代码抓手

代码块

```
1 import { ref, nextTick } from 'vue'
2 const visible = ref(false)
3 async function openPanel() {
4   visible.value = true
5   await nextTick()
6   document.querySelector('#panel')?.scrollIntoView()
7 }
```

高频追问与回答

1. `nextTick` 和 `setTimeout` 一样吗？

答：不一样。`setTimeout` 只是把任务丢到未来某个宏任务里，不关心 Vue 的更新队列；`nextTick` 绑定的是“等本轮 Vue 更新刷完”这个语义。

2. 为什么改了状态后立刻访问 DOM 还是旧的？

答：因为 Vue 会先把更新 job 入队，而不是每次赋值都同步 patch DOM。DOM 还没刷新完成时，你拿到的当然还是旧结构。

3. 什么场景下用 `flush: 'post'`，什么场景下用 `nextTick`？

答：如果你已经在 `watch/watchEffect` 里，且逻辑就是“等这一轮 DOM 更新后再执行”，`flush: 'post'` 很合适；如果你是在普通函数流里改完状态后再等 DOM 更新，则更常用 `nextTick`。

4. 为什么 Vue 要做调度器，而不是每改一次状态就立即更新？

答：因为同步直刷会导致大量重复工作，尤其是同一轮里连续多次改状态时。调度器通过批量、去重和稳定顺序来控制更新成本。

易错点

1. 把 `nextTick` 当通用异步工具。

2. 不需要 DOM 后状态时也机械使用 `nextTick`。

3. 不理解它和 scheduler 的关系，只把它当“保险写法”。

9.8 题目：一份 `.vue` 文件是怎么变成页面的？

考点

SFC、`compiler-sfc`、模板编译、render、VNode、runtime-core、patch。

3-5 年回答

`.vue` 文件不会直接在浏览器里执行，它会先被 Vue 的编译器处理。`template` 会被编译成 render 函数，`script setup` 会被编译成标准组件逻辑。运行时创建组件实例后执行 `setup` 和 `render`，`render` 生成 VNode，再由 Vue 把它挂载成真实 DOM。

5-7 年展开

更完整的链路是：SFC 先由 `compiler-sfc` 解析成 descriptor，拆出 `template`、`script`、`style`；`script setup` 在编译期展开；模板再走 `parse -> transform -> generate` 变成 render。运行时的 `runtime-core` 会创建组件实例、初始化 props 和 slots、执行 `setup`、建立 render effect。render 每次执行返回新 VNode，renderer 再通过 patch 把新旧 VNode 的差异同步到 DOM。Vue 真正执行的不是模板文本，而是编译后的 render 和运行时组件逻辑。

代码抓手

代码块

```
1 <template>
2   <button @click="count++">{{ count }}</button>
3 </template>
4 <script setup lang="ts">
5   import { ref } from 'vue'
6   const count = ref(0)
7 </script>
```

代码块

```
1 // 心智链路
2 SFC
3 -> compiler-sfc
4 -> compile template to render
5 -> runtime-core create component instance
6 -> render returns vnode
7 -> patch to DOM
```

高频追问与回答

1. `compiler-sfc` 和 `runtime-core` 分别做什么？

答：`compiler-sfc` 负责单文件组件解析和编译期处理；`runtime-core` 负责组件实例、VNode、render effect、scheduler、patch 等运行时主流程。

2. 为什么模板不是运行时字符串替换？

答：因为在构建型项目里，模板通常早就在编译阶段变成 render 了。运行时执行的是 render 生成 VNode，再通过 patch 更新 DOM。

3. `script setup` 为什么说是编译期语法糖？

答：因为它会在编译阶段展开成普通组件逻辑和 `setup` 语义，不是浏览器里多了一套新的运行时机制。

4. render 和 VNode 在更新链路里分别处于什么位置？

答：render 是“生成 UI 描述”的函数，VNode 是 render 的产物；patch 则负责把新旧 VNode 差异落到真实 DOM。

易错点

1. 把 `compiler-sfc` 说成 `runtime-sfc`。

2. 以为 `.vue` 文件浏览器可以直接运行。

3. 只会说“模板会变 render”，但接不上运行时组件更新链路。

9.9 题目：Vue 3 为什么通常比 Vue 2 快？

考点

响应式重构、编译优化、运行时精准更新、keyed diff、LIS。

3-5 年回答

Vue 3 更快不只是因为用了 `Proxy`。它在响应式、模板编译和渲染更新上都做了优化。比如 Vue 3 能更统一地处理对象新增属性、数组索引和复杂数据结构，模板编译阶段也会提前标记动态内容，运行时更新时就能少做很多无意义比较，所以整体性能通常更好。

5-7 年展开

完整回答最好拆成三层。第一层，响应式从 `Object.defineProperty` 升级到 `Proxy`，解决了 Vue 2 很多结构性限制。第二层，编译器会做静态提升、PatchFlag 标记、Block Tree 优化，把“哪里会变”提前告诉运行时。第三层，运行时在 keyed children 更新时会结合前后缀同步、中段映射和 LIS 来减少真实 DOM 移动。也就是说，Vue 3 更快不是一个点快，而是编译期、响应式层和运行时协同优化。

代码抓手

代码块

```
1 createElementVNode(  
2   'div',  
3   { class: _ctx.cls },  
4   _toDisplayString(_ctx.msg),  
5   3 /* TEXT | CLASS */  
6 )
```

高频追问与回答

1. 为什么“因为 Proxy”这个回答不完整？

答：因为 Proxy 只解释了响应式层的升级，解释不了编译期静态分析、PatchFlag、Block Tree 和运行时 diff 优化，所以它只是答案的一部分。

2. PatchFlag 和 Block Tree 分别解决什么问题？

答：PatchFlag 用来告诉运行时当前节点更新时重点关注什么，比如文本、class、props；Block Tree 则把动态后代节点收集成更容易遍历的 dynamicChildren，更新时可以跳过大量静态树。

3. Vue 3 的 diff 为什么能减少 DOM 移动？

答：因为它不会简单暴力地重排所有列表节点，而是先同步前后缀，再处理中段，并通过 LIS 保留那批已经相对有序的节点，只移动真正必须移动的部分。

4. Vue 2 和 Vue 3 的升级除了性能还有什么？

答：还有组件组织方式、TS 友好性、SFC 编译体验、SSR 能力和整体工程现代化。

易错点

1. 把 Vue 3 性能提升完全归因于 Proxy。
2. 不知道编译器在性能上的作用。
3. 只会说“虚拟 DOM 更快”，但讲不清为什么更新更精准。

9.10 题目：Vue 组件通信应该怎么选？

考点

单向数据流、props/emits、v-model、provide/inject、模板 ref、状态边界。

3-5 年回答

组件通信默认还是单向数据流。父传子用 props，子传父用 emit。受控输入类组件可以用 v-model。如果是跨很多层组件传递上下文，可以考虑 provide/inject。模板 ref 更适合访问

DOM 或少量子组件公开方法，不适合当成主要通信方式。

5-7 年展开

更成熟的回答应该强调“判断顺序”。先看 `props + emit` 能不能解决，因为这是边界最清晰的默认模型；如果是受控值输入能力，再考虑 `v-model`，但本质上它仍然是 `prop + emit` 协议；如果是深层组件树共享上下文，再考虑 `provide/inject`；只有极少量 imperative 能力，比如 `focus/open/reset`，才适合模板 ref。真正的重点不是 API 名字，而是如何守住单向数据流和边界清晰。

代码抓手

代码块

```
1  <script setup lang="ts">
2  const props = defineProps<{ modelValue: string }>()
3  const emit = defineEmits<{
4    'update:modelValue': [value: string]
5  }>()
6  </script>
7  <template>
8    <input
9      :value="props.modelValue"
10     @input="emit('update:modelValue', ($event.target as
HTMLInputElement).value)"
11   />
12 </template>
```

高频追问与回答

1. `defineModel()` 能不能替代所有 `props + emits`？

答：不能。它适合封装组件级受控输入能力，但普通业务协作默认还是 `props + emits` 更清晰，因为数据来源和更新路径更显式。

2. `provide/inject` 和 Pinia 怎么选？

答：`provide/inject` 更像局部上下文注入，适合组件树内部的深层共享；Pinia 更像应用级共享状态容器，适合跨页面、跨模块的共享状态。

3. 为什么不推荐大量使用组件实例 ref 做通信？

答：因为它会让父组件直接耦合子组件内部实现，边界会从声明式流转回命令式操控，长期维护成本更高。

4. 子组件为什么不能直接改 props？

答：因为 props 代表父组件传进来的输入，子组件直接改它会破坏单向数据流，也会让状态来源和责任边界变模糊。

易错点

1. 一遇到双向交互就条件反射上 `v-model`。
2. 一遇到深层传值就上全局 store。
3. 用模板 ref 替代正常的声明式通信。

9.11 题目：为什么 Pinia 常常要配合 `storeToRefs()` ？

考点

store 解构丢响应性、Pinia 与 Vue 响应式系统关系、SSR、setup store。

3-5 年回答

因为 Pinia store 本质上也是响应式对象，直接解构会有响应性丢失问题。`storeToRefs()` 可以把 store 里的 state 和 getters 转成 refs，解构后还能保持响应式。actions 不需要放进 `storeToRefs()`，直接从 store 上取就行。

5-7 年展开

`storeToRefs()` 的定位和 `toRefs()` 很像，但它是专门为 Pinia store 设计的。Pinia 官方文档明确说明，它会把 store 中的 state、getters 以及插件加上的响应式状态字段转成 refs，同时忽略 methods 和非响应式属性。之所以需要它，本质上还是因为 Pinia 建立在 Vue 响应式模型上，直接解构会绕开代理读取。SSR 场景下还要进一步注意，Pinia 应按请求创建独立实例，并在正确上下文里调用 `useStore()`。

代码抓手

代码块

```
1 import { storeToRefs } from 'pinia'
2 const userStore = useUserStore()
3 const { profile, isLogin } = storeToRefs(userStore)
4 const { logout } = userStore
```

高频追问与回答

1. 为什么 actions 不用 `storeToRefs()` ？

答：因为 actions 本来就是方法，不存在“解构后失去响应性”的问题。`storeToRefs()` 的目标是状态和 getters，不是方法。

2. `storeToRefs()` 和 `toRefs()` 有什么区别？

答：`storeToRefs()` 是为 Pinia store 定制的，会自动忽略方法和非响应式字段；`toRefs()` 是通用的响应式对象工具，不懂 Pinia store 的特殊结构。

3. Pinia 在 SSR 里要注意什么？

答：要保证每个请求都有独立 pinia 实例，避免跨请求状态污染；在组件外部使用 store 时，还要显式传入 pinia 实例。

4. 为什么说 Pinia 更符合 Vue 3？

答：因为它的建模方式更贴近 Composition API 和 Vue 响应式系统，TS 推断也更自然，不像 Vuex 那样更强调模板化约束。

易错点

1. 直接解构 store 里的状态字段。

2. 把 methods 也塞进 `storeToRefs()`。

3. 在 SSR 场景里把 pinia 写成进程级共享单例。

9.12 题目：SSR、CSR、Hydration 到底怎么理解？

考点

首屏渲染模式、服务端输出、客户端激活、mismatch、跨请求状态污染。

3-5 年回答

CSR 是服务端先返回一个壳 HTML，客户端下载 JS 后再生成页面；SSR 是服务端先把页面 HTML 渲染出来返回给浏览器，客户端再把事件和交互接上去。Hydration 就是把客户端逻辑接管服务端已经生成的 DOM，而不是重新全量创建一遍。

5-7 年展开

真正拉开差距的是两个问题。第一，hydration mismatch。它的本质是服务端输出的 HTML 结构和客户端首屏 vnode 结构不一致，常见来源包括随机数、时间戳、非法 HTML、浏览器专属 API、服务端和客户端初始状态不一致。第二，跨请求状态污染。服务端进程通常是常驻的，如果 app、router、pinia 或共享状态写成模块级单例，一个用户请求写进去的数据就可能被另一个用户读到。真正做过 SSR 的人，回答不会只停留在“SSR 首屏更快”。

代码抓手

代码块

```
1 import { createSSRApp } from 'vue'
2 import { createPinia } from 'pinia'
```

```
3 export function createApp() {
4   const app = createSSRApp(App)
5   const pinia = createPinia()
6   const router = createRouter(...)
7   app.use(pinia)
8   app.use(router)
9   return { app, pinia, router }
10 }
```

高频追问与回答

1. Hydration 是不是重新渲染一遍页面？

答：不是。它的目标是接管已有 DOM，把事件监听、组件实例和响应式系统接到服务端输出的 HTML 上。

2. 什么最容易引发 hydration mismatch？

答：首屏状态不一致、随机数和时间戳、直接访问 `window/document`、非法 HTML 结构，这几类最常见。

3. 为什么 SSR 一定要强调“每请求独立实例”？

答：因为服务端模块通常只初始化一次。如果共享状态是单例，不同请求之间就可能串数据，这是 SSR 非常典型也非常严重的问题。

4. Teleport 在 SSR 下有什么额外注意点？

答：它不会像普通节点那样自然出现在主应用 HTML 字符串里，服务端渲染时要考虑 teleports 的插入位置和激活一致性。

易错点

1. 把 hydration 理解成“客户端再重新渲染一遍”。

2. 只讲 SSR 好处，不讲一致性治理成本。

3. 忽略每请求独立实例这个核心原则。

9.13 题目：Vue 性能优化在工程里应该怎么拆？

考点

加载性能、更新性能、架构选型、边界稳定性、列表和响应式开销。

3-5 年回答

我会先分成加载性能和更新性能。加载性能重点看路由懒加载、重型组件异步加载、包体积控制；更新性能重点看列表渲染、`key` 是否稳定、`v-if` 和 `v-show` 是否选对、有没有不必要的深监听和大对象响应式开销。

5-7 年展开

更成熟的拆法是三层。第一层看架构：CSR、SSR、SSG 选型是否合理，首屏是不是被过量 JS 拖慢。第二层看边界：props 是否稳定、组件粒度是否合理、共享状态是否过大、是否把本地状态错误上提。第三层看局部策略：路由懒加载、异步组件、虚拟列表、`v-once`、`v-memo`、`shallowRef`、`markRaw` 等。真正的大多数性能问题，不是 Vue 不够快，而是架构、边界和状态设计没有收住。

代码抓手

代码块

```
1 import { shallowRef } from 'vue'
2 const bigGraph = shallowRef<any[]>([])
3 function replaceGraph(next: any[]) {
4   bigGraph.value = next
5 }
```

高频追问与回答

1. 性能优化为什么不能只背技巧清单？

答：因为不同技巧解决的是不同层次的问题。首屏慢可能是架构和拆包问题，更新慢可能是边界和状态建模问题，混着优化常常抓不到主因。

2. 为什么说 props 稳定性很重要？

答：因为子组件是否需要更新，首先取决于它的输入是否变化。父组件每次都传新对象、新数组、新函数，就会扩大更新范围。

3. 大列表性能为什么往往不是 diff 算法本身的问题？

答：因为大量真实 DOM 才是更直接的瓶颈。真正有效的方案通常是虚拟滚动、稳定 key、减少节点层级，而不是只靠框架 diff。

4. 大对象和第三方实例为什么要控制响应式边界？

答：因为深响应不是免费的。图表实例、编辑器实例、大型不可变结构如果无脑深代理，会增加追踪成本和理解成本，通常更适合 `shallowRef`、`markRaw` 这类方案。

易错点

1. 没测量就优化。
2. 把组件更新本身当 bug，而不是先判断更新是否合理。
3. 只会讲局部技巧，不会先判断瓶颈在加载还是更新。